

Contents

1 Aurnob	2	3 Sazid	10
1.1 A_Formulas	2	3.1 Articulation_Bridge	10
1.1.1 Combinatorics	2	3.2 Articulation_Point	10
1.1.2 Fibonacci	2		
1.1.3 NT	3	4 Tamzid	10
1.1.4 bitwise	3	4.1 bellmanford	10
1.2 Build	3	4.2 moalgo	10
1.2.1 OJ	3	4.3 sckkosaraju	11
1.2.2 sublime	3	4.4 sparsetable	11
1.3 Geometry	3	4.5 treediameter	11
1.3.1 Angle between 3 points	3		
1.3.2 MEC	3	5 D-One	11
1.3.3 Rotating_Calipers	3	5.1 01_template	11
1.3.4 geometry general	4	5.2 02_lamda	11
1.4 Graph	5	5.3 03_CheckFunctions	11
1.4.1 MST Kruskal's	5	5.4 04_import_Mod	11
1.4.2 MST Prim's	5	5.5 05_Comperator	12
1.4.3 Top Sort	5	5.6 06_Number_theory	12
1.5 Math	5	5.7 07_nCr	12
1.5.1 Binary Operations	5	5.8 08_Binary_Search	12
1.5.2 Gaussian Elimination	5	5.9 09_Kadane	12
1.5.3 Matrix Exponentiation	5	5.10 10_2DPrefixSum	12
1.5.4 Mobius Precomputation	6	5.11 11_dfsTree	12
1.5.5 Modular Inverse	6	5.12 12_dfsGraph	12
1.5.6 Nth Root Floor	6	5.13 13_lca_blift	12
1.5.7 Sieve	6	5.14 14_bfs	13
1.5.8 Totient Values	6	5.15 15_dijkstra	13
1.6 Segment Tree	6	5.16 16_dsu	13
1.6.1 Persistent Segtree	6	5.17 17_ordered_set	13
1.6.2 Segment Tree Merge Sort Tree	6	5.18 18_segTree_node	13
1.6.3 Segment Tree with Lazy Propagation	7	5.19 19_fenwick	14
1.6.4 SegmentTree	7	5.20 20_rangefenwick	14
1.7 String	7		
1.7.1 Double String Hash	7	6 zFrom Sifat Bhai	14
1.7.2 KMP and Z Function	7	6.1 Data Structure	14
1.7.3 Suffix Array	8	6.1.1 persistant_segtree	14
1.7.4 Trie	8	6.2 Geometry	14
1.8 CUSTOM HASH	8	6.2.1 3D	14
1.9 Coordinate Compression	8	6.2.1.1 3D Convex Hull	14
1.10 Custom_Comparator_for_pq	8	6.2.2 Circle	15
1.11 RMQ	9	6.2.2.1 Circle Intersection	15
1.12 Subtree queries - Number of nodes with color k in the subtree	9	6.2.2.2 Circle Polygon Intersection	15
1.13 bitset_operations	9	6.2.2.3 Circle Tangents	15
1.14 next permutation	9	6.2.3 Polygon	15
		6.2.3.1 Hull Diameter	15
2 Kabya	9	6.2.3.2 Line Hull Intersection	15
2.1 crt	9	6.2.3.3 Polygon Center	16
2.2 extendedEuclidean	9	6.3 Graph	16
2.3 lcs	9	6.3.1 2Sat	16
2.4 millerRabin	9	6.4 Math	16
2.5 zzsublime_build	10	6.4.1 FloorSum	16
		6.4.2 catalan	16
		6.4.3 gen_all_k_combs	16
		6.4.4 next_lexicographical_k_comb	16
		6.4.5 ntt	16
		6.4.6 xor_basis	17

1 Aurnob

1.1 A_Formulas

1.1.1 Combinatorics

- $\sum_{k=0}^n \binom{n-k}{k} = \text{Fib}_{n+1}$
- $\binom{n}{k} = \binom{n}{n-k}$
- $\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$
- $k \binom{n}{k} = n \binom{n-1}{k-1}$
- $\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1}$
- $\sum_{i=0}^n \binom{n}{i} = 2^n$
- $\sum_{i=0}^{\lfloor n/2 \rfloor} \binom{n}{2i} = 2^{n-1}$
- $\sum_{i=0}^{\lfloor (n-1)/2 \rfloor} \binom{n}{2i+1} = 2^{n-1}$
- $\sum_{i=0}^k (-1)^i \binom{n}{i} = (-1)^k \binom{n-1}{k}$
- $\sum_{i=0}^k \binom{n+i}{i} = \binom{n+k+1}{k}$
- $\sum_{i=1}^n i \binom{n}{i} = n \cdot 2^{n-1}$
- $\sum_{i=1}^n i^2 \binom{n}{i} = (n+n^2) \cdot 2^{n-2}$
- Vandermonde's Identity: $\sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}$
- Hockey-Stick Identity: $\sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}$

- $\sum_{i=0}^k \binom{k}{i}^2 = \binom{2k}{k}$
- $\sum_{k=0}^n \binom{n}{k} \binom{n}{n-k} = \binom{2n}{n}$
- $\sum_{k=q}^n \binom{n}{k} \binom{k}{q} = 2^{n-q} \binom{n}{q}$
- $\sum_{i=0}^n k^i \binom{n}{i} = (k+1)^n$
- $\sum_{i=0}^n \binom{2n}{i} = 2^{2n-1} + \frac{1}{2} \binom{2n}{n}$
- $\sum_{i=1}^n \binom{n}{i} \binom{n-1}{i-1} = \binom{2n-1}{n-1}$
- $\sum_{i=0}^n \binom{2n}{i}^2 = \frac{1}{2} \left(\binom{4n}{2n} + \binom{2n}{n}^2 \right)$
- Catalan Number: $C_n = \frac{1}{n+1} \binom{2n}{n}$
- Derangement: $d(n) = (n-1) \cdot (d(n-1) + d(n-2))$, with $d(0) = 1, d(1) = 0$
- Stirling Numbers of the First Kind: $s(n, k) = s(n-1, k-1) + (n-1) \cdot s(n-1, k)$
- Stirling Numbers of the Second Kind: $S(n, k) = S(n-1, k-1) + k \cdot S(n-1, k)$
- Bell Numbers: $B_{n+1} = \sum_{k=0}^n \binom{n}{k} B_k$
- $\sum_{i=0}^n i \cdot i! = (n+1)! - 1$
- $\sum_{i=1}^n i a^i = \frac{a(na^{n+1} - (n+1)a^n + 1)}{(a-1)^2}$
- Fibonacci: $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$
- $F_n = \sum_{k=0}^{\lfloor (n-1)/2 \rfloor} \binom{n-k-1}{k}$
- $F_n = \frac{1}{\sqrt{5}} \left(\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right)$

- $\sum_{i=1}^n F_i = F_{n+2} - 1$

- $\sum_{i=0}^{n-1} F_{2i+1} = F_{2n}$

- $\sum_{i=1}^n F_{2i} = F_{2n+1} - 1$

• **Pick's Theorem:** For a simple polygon with vertices on lattice points, $A = I + \frac{B}{2} - 1$ where A is area, I is number of interior lattice points, and B is number of boundary lattice points.

1.1.2 Fibonacci

Let A, B and n be integer numbers.

$$k = A - B$$

$$F_A F_B = F_{k+1} F_A^2 + F_k F_A F_{A-1}$$

$$\sum_{i=0}^n F_i^2 = F_{n+1} F_n$$

$$\sum_{i=0}^n F_i F_{i+1} = F_{n+1}^2 - (-1)^n$$

$$\sum_{i=0}^n F_i F_{i-1} = \sum_{i=0}^{n-1} F_i F_{i+1}$$

$$\gcd(F_m, F_n) = F_{\gcd(m, n)}$$

$$\sum_{0 \leq k \leq n} \binom{n-k}{k} = F_{n+1}$$

$$\gcd(F_n, F_{n+1}) = \gcd(F_n, F_{n+2}) = \gcd(F_{n+1}, F_{n+2}) = 1$$

1.1.3 NT

$$\sum_{k=1}^n \frac{1}{\gcd(k, n)} = \sum_{d|n} \frac{1}{d} \varphi\left(\frac{n}{d}\right) = \frac{1}{n} \sum_{d|n} d \varphi(d)$$

$$\sum_{k=1}^n \frac{k}{\gcd(k, n)} = 1 + \frac{1}{2} \sum_{\substack{d|n \\ d>1}} d \varphi(d)$$

$$\sum_{k=1}^n \frac{n}{\gcd(k, n)} = 2 \sum_{k=1}^n \gcd(k, n) - 1, \quad \text{for } n > 1$$

$$\sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = 1] = \sum_{d=1}^n \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2 = 2 \sum_{i=1}^n \varphi(i) - 1$$

$$\sum_{i=1}^n \sum_{j=1}^n \gcd(i, j) = \sum_{d=1}^n \varphi(d) \left\lfloor \frac{n}{d} \right\rfloor^2$$

$$\sum_{i=1}^n \sum_{j=1}^n i \cdot j [\gcd(i, j) = 1] = \sum_{d=1}^n \mu(d) \cdot d^2 \left(\frac{\lfloor n/d \rfloor (\lfloor n/d \rfloor + 1)}{2} \right)^2$$

$$\sum_{i=1}^n \sum_{j=1}^n \text{lcm}(i, j) = \sum_{d=1}^n d \sum_{k=1}^{\lfloor n/d \rfloor} \mu(k) \cdot k^2 \left(\frac{\lfloor \frac{n}{d \cdot k} \rfloor (\lfloor \frac{n}{d \cdot k} \rfloor + 1)}{2} \right)^2$$

$$\sum_{d|n} \mu(d) = [n = 1]$$

$$\sum_{d|n} \varphi(d) = n$$

$$\sum_{d|n} \mu(d) \frac{n}{d} = \varphi(n)$$

$$g(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d) g\left(\frac{n}{d}\right)$$

$$g(n) = \sum_{k=1}^{\lfloor M/n \rfloor} f(k \cdot n) \iff f(n) = \sum_{k=1}^{\lfloor M/n \rfloor} \mu(k) g(k \cdot n)$$

$$\sum_{i=1}^n \gcd(i, n) = \sum_{d|n} d \cdot \varphi\left(\frac{n}{d}\right)$$

$$\sum_{i=1}^n \text{lcm}(i, n) = \frac{n}{2} \left(\sum_{d|n} d \cdot \varphi(d) + 1 \right)$$

$$\sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = 1] = \sum_{d=1}^{\min(n, m)} \mu(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor$$

$$\sum_{i=1}^n \sum_{j=1}^m \gcd(i, j) = \sum_{d=1}^{\min(n, m)} \varphi(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor$$

$$v_p(n!) = \sum_{k=1}^{\infty} \left\lfloor \frac{n}{p^k} \right\rfloor$$

$$\binom{n}{k} \equiv \prod_{i=0}^r \binom{n_i}{k_i} \pmod{p}$$

1.1.4 bitwise

$$\bullet a + b = a \oplus b + 2 \cdot (a \& b)$$

$$\bullet a + b = a | b + a \& b$$

$$\bullet a \oplus b = a | b - a \& b$$

$$\bullet n \bmod 2^i = n \& (2^i - 1)$$

• The k -th bit is set in x iff:

$$x \bmod (2^{k+1} - 1) \geq 2^k$$

• Equivalently, k -th bit is set iff:

$$x \bmod (2^{k+1} - 1) - x \bmod 2^k \neq 0$$

(The result will be exactly 2^k if set.)

$$\bullet 1 \oplus 2 \oplus 3 \oplus \dots \oplus (4k - 1) = 0 \quad \text{for any } k \geq 0$$

1.2 Build

1.2.1 OJ

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    #ifndef ONLINE_JUDGE
    freopen("input.txt", "r", stdin);
    freopen("output.txt", "w", stdout);
    #endif
    int n; cin >> n;
    for(int i=0; i<n; i++){
        cout << i+1 << endl;
    }
    cout << "hello" << endl;
}
```

1.2.2 sublime

```
{
"cmd" : ["g++ -std=c++14 $file_name -o $file_base_name &&
         timeout 4s ./ $file_base_name<input.txt>output.txt"],
"selector" : "source.c",
"shell": true,
"working_dir" : "$file_path"
}
```

1.3 Geometry

1.3.1 Angle between 3 points

```
double angledeu(pt a, pt b, pt c){
    pll ba={b.x-a.x, b.y-a.y};
    pll bc={b.x-c.x, b.y-c.y};
    double BA=atan2((double)ba.sc, (double)ba.ft);
    double BC=atan2((double)bc.sc, (double)bc.ft);
    double rslt = BA - BC;
    double rs = (rslt * (double)180.00) / pi;
    if(rs>=0){
        if(rs>180.00){
            return double(360.00-rs);
        }else{
            return rs;
        }
    }else{
        return double(rs*(double(-1.00)));
    }
}
```

1.3.2 MEC

```
// Given n points, return the radius of the minimum enclosing
// circle
// Call convex_hull() before this for faster solution
// Expected O(n)
double minimum_enclosing_circle_radius(vector<pointdb> &p) {
    random_shuffle(p.begin(), p.end());
    int n = p.size();
    pointdb center = p[0]; double radius = 0;
    for (int i = 1; i < n; i++) {
        if (DIST(center, p[i]) > radius + EPS) {
            center = p[i]; radius = 0;
            for (int j = 0; j < i; j++) {
                if (DIST(center, p[j]) > radius + EPS) {
                    center = (p[i] + p[j]) / 2.0;
                    radius = DIST(p[i], p[j]) / 2.0;
                    for (int k = 0; k < j; k++) {
                        if (DIST(center, p[k]) > radius + EPS) {
                            pointdb a = p[i], b = p[j], c = p[k];
                            pointdb ab = (a + b) / 2.0, ac = (a + c) / 2.0;
                            pointdb dab = (b - a) * pointdb(0, 1);
                            pointdb dac = (c - a) * pointdb(0, 1);
                            double t = ((ac - ab).x * dab.y - (ac - ab).y * dab.x) /
                                ((dab.x * dac.y - dab.y * dac.x));
                            center = ab + dab * t;
                            radius = DIST(center, a);
                        }
                    }
                }
            }
        }
    }
    return radius;
}
```

1.3.3 Rotating Calipers

```
// cross(o, a, b) = (a - o) X (b - o)
ll cross(const point &o, const point &a, const point &b) {
    return (a.x - o.x) * (b.y - o.y)
        - (a.y - o.y) * (b.x - o.x);
}
// Rotating calipers to find max squared distance on convex
// polygon H
ll max_pair_dist2(const vector<point> &H) {
    int m = H.size();
    if (m < 2) return 0;
    if (m == 2) return DIST2(H[0], H[1]);
    ll best = 0; int j = 1;
    for (int i = 0; i < m; ++i) {
        while (abs(cross(H[i], H[(i+1)%m], H[(j+1)%m]))
            > abs(cross(H[i], H[(i+1)%m], H[j]))) {
            j = (j+1) % m;
        }
        best = max(best, (ll)DIST2(H[i], H[j]));
        best = max(best, (ll)DIST2(H[(i+1)%m], H[j]));
    }
    return best;
}
```

1.3.4 geometry general

```
#include<bits/stdc++.h>
#include<ext/pb_ds/assoc_container.hpp>
#include<ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

typedef long long int ll;
typedef vector<long long int> vll;
typedef pair<long long, long long> pll;
typedef long double lld;
typedef tree<int, null_type, less_equal<int>,
            rb_tree_tag,
            tree_order_statistics_node_update> pbds;

#define loop(i,k,n) for(ll i=k;i<n;i++)
#define for_(item,vec) for(auto item:vec)
#define ft first
#define sc second
#define pb push back
#define sz(x) ((int)(x).size())
#define yes cout<<"Yes"<<endl
#define no cout<<"No"<<endl
#define Yes cout<<"YES"<<endl
#define No cout<<"NO"<<endl
#define Alice cout<<"Alice"<<endl
#define Bob cout<<"Bob"<<endl
#define newl cout<<"\n"
#define clean fflush(stdout)
#define all(x) (x).begin(),(x).end()
#define inpar(arr,n) ll arr[n]; loop(i,0,n) cin
>>arr[i]
#define invvec(v,n) vector<ll> v(n); for(auto
&i:v) cin>>i
#define CEIL(x,y) ((x+y-1)/y)
#define set_bits builtin_popcountll
#define PI 3.141592653589793238462
#define MOD 1000000007ll
// #define MOD1 998244353ll
#define INF 1e18
#define ONLINE_JUDGE
#define debug(x) cerr << #x << " "; _print(x); cerr << endl;
#define else
#define debug(x)
#define endif

const ll N=5e6+20;
const ll MOD1=998244353;

const double eps=1e-9;
int sign(double x){return (x>eps)-(x<-eps);}
typedef long long T;
struct pt{
    T x,y;
    pt(){x=0,y=0;}
    pt(T a,T b):x(a),y(b){}
    pt operator+(pt p){return {x+p.x,y+p.y};}
    pt operator-(pt p){return {x-p.x,y-p.y};}
    pt operator-(const pt &a) const{return pt(x-a.x,y-a.y);}
    pt operator*(T d){return {x*d,y*d};}
    pt operator/(T d){return {x/d,y/d};}
    bool operator==(pt a) const{return sign(a.x-x)==0 && sign
(a.y-y)==0;}
    bool operator!=(pt a) const{return !(this==a);}
    friend double NORM(pt a){return sqrt((double)(a.x*a.x+a.y
*a.y));}
    friend T NORM2(pt a){return a.x*a.x+a.y*a.y;}
    friend istream& operator>>(istream&in,pt &p){return in>>p
.x>>p.y;}
    friend ostream& operator<<(ostream&os,pt p){return os<<"(
"<<p.x<<","<<p.y<<")";}
    double arg(){return atan2(y,x);}
};
inline T dot(pt a,pt b){return a.x*b.x+a.y*b.y;}
inline T dist2(pt a,pt b){return NORM2(a-b);}
```

```
inline double dist(pt a,pt b){return NORM(a-b);}
inline T cross(pt a,pt b){return a.x*b.y-a.y*b.x;}
inline T cross3(pt a,pt b,pt c){return cross(b-a,c-a);}
inline int orient(pt a,pt b,pt c){return sign(cross(b-a,c-a))
;}
inline double angle(pt v, pt w) {
    double cosTheta = dot(v, w) / NORM(v) / NORM(w);
    return acos(max(-1.0, min(1.0, cosTheta)));
}
inline double rad to deg(double r) {return (r*180.0/PI);}
inline double deg to rad(double d) {return (d*PI/180.0);}
bool inDisk(pt a,pt b,pt p){return dot(a-p,b-p)<=0;}
// p is in the circle with diameter AB
bool onSegment(pt a,pt b,pt p){return orient(a,b,p)==0 &&
inDisk(a,b,p);}
double areaPolygon(vector<pt>&p){
    double area=0.0;
    for(int i=0,n=p.size();i<n;i++){
        area+=cross(p[i],p[(i+1)%n]);
    }
    return abs(area)/2.0;}
bool inPolygon(vector<pt> p, pt a, bool strict = true){
    auto above=[](pt a, pt p){return p.y>=a.y;};
    auto crossesRay=&[(pt a, pt p, pt q) {
        return (above(a,q) - above(a,p)) * orient(a,p,q) > 0;
    }];
    int numCrossings = 0;
    for (int i = 0, n = p.size(); i < n; i++){
        if(onSegment(p[i],p[(i+1)%n],a)) return !strict;
        numCrossings += crossesRay(a, p[i], p[(i + 1) % n]);
    }
    return numCrossings & 1;
}
pt perp(pt a) {return pt{-a.y,a.x};}
bool inAngle(pt a,pt b,pt c,pt p){//p lies in the angle <BAC(
CW)
    assert(orient(a,b,c)!=0);
    if(orient(a,b,c)<0) swap(b,c);
    return orient(a,b,p)>=0 && orient(a,c,p)<=0;
}
double orientedAngle(pt a,pt b,pt c){//CW BAC
    if(orient(a,b,c)>=0)return angle(b-a,c-a);
    else return 2.0*PI-angle(b-a,c-a);
}
bool isConvex(vector<pt>&p){//collection of points makes a
convex hull
    bool hasPos=false,hasNeg=false;
    for (int i=0,n=p.size();i<n;i++){
        int o=orient(p[i],p[(i+1)%n],p[(i+2)%n]);
        if(o>0)hasPos = true;
        if(o<0)hasNeg = true;
    }
    return !(hasPos && hasNeg);
}
struct line{
    pt v;T c;
    line(pt v,T c):v(v),c(c){}
    line(T a,T b,T c):v({b,-a}),c(c){}
    line(pt p,pt q):v(q-p),c(cross(v,p)){}
    T side(pt p) {return cross(v,p)-c;}//neg = rightside of
vector PQ
    double dist(pt p) {return abs(side(p))/NORM(v);}
    double sqDist(pt p) {return side(p)*side(p)/NORM2(v);}
    // line that is perpendicular to this and goes through
point p
    line perpThrough(pt p) {return {p+perp(v)};}
    bool cmpProj(pt p,pt q) {return dot(v,p)<dot(v,q);}
    bool intersection(line l1, line l2, pt &out) {
        T d = cross(l1.v, l2.v);
        if (d == 0) return false;
        out=(l2.v*l1.c-l1.v*l2.c)/d;
        return true;
    }
    void shiftLeft(double dist);
    pt proj(pt p) {return p-perp(v)*side(p)/NORM2(v);}
    pt refl(pt p) {return p-perp(v)*2*side(p)/NORM2(v);}
```

```
struct pt3d{
    T x, y, z;
    pt3d operator+(pt3d p){return {x+p.x,y+p.y,z+p.z};}
    pt3d operator-(pt3d p){return {x-p.x,y-p.y,z-p.z};}
    pt3d operator*(T d){return {x*d,y*d,z*d};}
    pt3d operator/(T d){return {x/d,y/d,z/d};}
    bool operator==(pt3d p){return tie(x,y,z)==tie(p.x,p.y,p
.z);}
    bool operator!=(pt3d p){return !operator==(p);}
    friend T operator|(pt3d v,pt3d w){return v.x*w.x+v.y*w.y+
v.z*w.z;}
    friend pt3d operator*(pt3d v, pt3d w) {
        return {v.y*w.z-v.z*w.y,
                v.z*w.x-v.x*w.z,
                v.x*w.y-v.y*w.x};
    }
};
T sq(pt3d v) {return v|v;}
double NORM3(pt3d v) {return sqrt(sq(v));}
pt3d unit(pt3d v) {return v/NORM3(v);}
double angle(pt3d v, pt3d w) {
    double cosTheta=(v|w)/NORM3(v)/NORM3(w);
    return acos(max(-1.0,min(1.0,cosTheta)));
}
T orient(pt3d p,pt3d q,pt3d r,pt3d s){
    return (q-p)*(r-p)|(s-p);//+ve -> s above pqr
}
void convex_hull(vector<pt>&a, bool incl_col=0) {
    pt p0 = *min_element(a.begin(),a.end(),[](pt a,pt b){
        return make_pair(a.y,a.x)<make_pair(b.y,b.x);
    });
    sort(a.begin(), a.end(), [&p0](const pt&a,const pt&b){
        int o=orient(p0,a,b);
        if (o==0)
            return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y)*(p0.y-a
.y)
                < (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)*(p0.y-b
.y);
        return o < 0;
    });
    if (incl_col){
        int i=(int)a.size()-1;
        while (i>=0 && orient(p0,a[i],a.back())==0) i--;
        reverse(a.begin()+i+1,a.end());
    }
    auto cw=[](pt a, pt b, pt c,bool incl_col) {
        int o = orient(a, b, c);
        return o < 0 || (incl_col && o == 0);
    };
    vector<pt> st;
    for (int i = 0; i < (int)a.size(); i++) {
        while (st.size() > 1 && !cw(st[st.size()-2], st.back
(), a[i], incl_col))
            st.pop_back();
        st.push_back(a[i]);
    }
    if (incl_col == false && st.size() == 2 && st[0] == st
[1]) st.pop_back();
    a = st;
}
struct HP {
    pt a, b;
    HP() {}
    HP(pt a, pt b) : a(a), b(b) {}
    HP(const HP& rhs) : a(rhs.a), b(rhs.b) {}
    int operator < (const HP& rhs) const {
        pt p = b-a;
        pt q = rhs.b - rhs.a;
        int fp = (p.y < 0 || (p.y == 0 && p.x < 0));
        int fq = (q.y < 0 || (q.y == 0 && q.x < 0));
        if (fp != fq) return fp == 0;
        if (cross(p, q)) return cross(p, q) > 0;
        return cross(p, rhs.b - a) < 0;
    }
}
pt line_line_intersection(pt a, pt b, pt c, pt d) {
    b = b - a; d = c - d; c = c - a;
```

```

    return a + b * cross(c, d) / cross(b, d);
}
pt intersection(const HP &v) {
    return line_line_intersection(a, b, v.a, v.b);
}
};
int check(HP a, HP b, HP c) {
    return cross(a.b - a.a, b.intersection(c) - a.a) > -eps;
    // -eps to include polygons of zero area (straight lines, points)
}
// consider half-plane of counter-clockwise side of each line
// if lines are not bounded add infinity rectangle
// returns a convex polygon, a point can occur multiple times though
// complexity: O(n log(n))
vector<pt> half_plane_intersection(vector<HP> h) {
    sort(h.begin(), h.end());
    vector<HP> tmp;
    for (int i = 0; i < h.size(); i++) {
        if (!i || cross(h[i].b - h[i].a, h[i - 1].b - h[i - 1].a)) {
            tmp.push_back(h[i]);
        }
    }
    h = tmp;
    vector<HP> q(h.size() + 10);
    int qh = 0, qe = 0;
    for (int i = 0; i < h.size(); i++) {
        while (qh - qe > 1 && !check(h[i], q[qe - 2], q[qe - 1])) qe--;
        while (qh - qe > 1 && !check(h[i], q[qh], q[qh + 1])) qh++;
        q[qe++] = h[i];
    }
    while (qh - qe > 2 && !check(q[qh], q[qe - 2], q[qe - 1])) qe--;
    while (qh - qe > 2 && !check(q[qe - 1], q[qh], q[qh + 1])) qh++;
    vector<HP> res;
    for (int i = qh; i < qe; i++) res.push_back(q[i]);
    vector<pt> hull;
    if (res.size() > 2) {
        for (int i = 0; i < res.size(); i++) {
            hull.push_back(res[i].intersection(res[(i + 1) % ((int)res.size())]));
        }
    }
    return hull;
}

```

1.4 Graph

1.4.1 MST Kruskal's

```

void solve() {
    ll n, m; cin >> n >> m;
    vector<pair<ll, pair<ll, ll>>> edges;
    loop(i, 1, n + 1) make_set(i);
    loop(i, 0, m) {
        ll u, v, wt;
        cin >> u >> v >> wt;
        edges.push_back({wt, {u, v}});
    }
    ll ans = 0; sort(edges.begin(), edges.end());
    loop(i, 0, m) {
        auto bruh = edges[i];
        ll u = bruh.second.first;
        ll v = bruh.second.second;
        ll wt = bruh.first;
        if (find_set(u) == find_set(v)) continue;
        union_sets(u, v); ans += wt;
    }
    set<ll> s;
    loop(i, 1, n + 1) s.insert(find_set(i));
    if (s.size() != 1) {
        cout << "IMPOSSIBLE\n"; return;
    }
    cout << ans << endl;
}

```

1.4.2 MST Prim's

```

void solve() {
    ll n, m;
    cin >> n >> m;
    priority_queue<pair<ll, ll>, vector<pair<ll, ll>>, greater<pair<ll, ll>>> pq;
    loop(i, 0, m) {
        ll u, v, wt; cin >> u >> v >> wt;
        graph[u].push_back({v, wt});
        graph[v].push_back({u, wt});
    }
    ll totalcost = 0; ll count = 0;
    pq.push({0, 1});
    while (!pq.empty() && count < n) {
        auto [cost, node] = pq.top();
        pq.pop();
        if (vis[node]) continue;
        vis[node] = true; totalcost += cost;
        count++;
        for (auto [child, wt] : graph[node]) {
            if (vis[child]) continue;
            pq.push({wt, child});
        }
    }
    if (count == n) cout << totalcost << endl;
    else cout << "IMPOSSIBLE\n";
}

```

1.4.3 Top Sort

```

/**
 * TOPOLOGICAL SORT (Kahn's Algorithm - BFS)
 * TIME COMPLEXITY: O(V + E)
 * SPACE COMPLEXITY: O(V + E)
 * USE CASE: Order tasks with dependencies. Detect cycles in directed graphs.
 * INPUT: n (nodes, 1-indexed here), adj (adjacency list)
 * OUTPUT: Vector with order. If output size != n, a CYCLE exists!
 */
vector<int> topoSort(int n, const vector<vector<int>>& adj) {
    vector<int> in_degree(n + 1, 0);
    // Step 1: Calculate in-degrees
    for (int u = 1; u <= n; ++u) {
        for (int v : adj[u]) {
            in_degree[v]++;
        }
    }
    // Step 2: Push all nodes with 0 in-degree to queue
    queue<int> q;
    for (int i = 1; i <= n; ++i) {
        if (in_degree[i] == 0) {
            q.push(i);
        }
    }
    // Step 3: Process nodes
    vector<int> order;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (int v : adj[u]) {
            in_degree[v]--;
            if (in_degree[v] == 0) {
                q.push(v);
            }
        }
    }
    // Step 4: Cycle Check
    if (order.size() != n) {
        return {}; // Returns empty vector if a cycle exists (impossible to sort)
    }
    return order;
}

```

1.5 Math

1.5.1 Binary Operations

```

ll number of set bits(ll n) {
    return __builtin_popcountll(n);
}

inline ll msb_idx(ll n) {
    if (n == 0) return -1;
}

```

```

return 63 - __builtin_clzll(n);
}

```

```

inline ll lsb_idx(ll n) {
    if (n == 0) return -1;
    return __builtin_ctzll(n);
}

```

```

inline bool is power of two(ll n) {
    return (n > 0) && ((n & (n - 1)) == 0);
}

```

```

vector<ll> decimal_to_binary(ll n) {
    vll ans; ll idx = 0;
    while (n > 0) {
        ans.pb(n % 2); n >>= 1;
    }
    while (ans.size() < 32) ans.pb(0);
    reverse(ans.begin(), ans.end());
    return ans;
}

```

1.5.2 Gaussian Elimination

```

// formation of the equation vector:
// [ 1 2 1 | 6 ] -> Represents x + 2y + z = 6
// [ 2 3 1 | 9 ] -> Represents 2x + 3y + z = 9
// [ 3 1 2 | 8 ] -> Represents 3x + y + 2z = 8
// Time Complexity : O(min(n,m)*n*m)
const double EPS = 1e-9;
const int INF = 2;
int gauss (vector < vector<double> > a, vector<double> & ans) {
    int n = (int) a.size();
    int m = (int) a[0].size() - 1;
    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n; ++col) {
        int sel = row;
        for (int i = row; i < n; ++i)
            if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
        if (abs(a[sel][col]) < EPS) continue;
        for (int i = col; i < m; ++i) swap(a[sel][i], a[row][i]);
        where[col] = row;
        for (int i = 0; i < n; ++i)
            if (i != row) {
                double c = a[i][col] / a[sel][col];
                for (int j = col; j < m; ++j)
                    a[i][j] -= a[sel][j] * c;
                ++row;
            }
        ans.assign(m, 0);
        for (int i = 0; i < m; ++i)
            if (where[i] != -1)
                ans[i] = a[where[i]][m] / a[where[i]][i];
        for (int i = 0; i < n; ++i) {
            double sum = 0;
            for (int j = 0; j < m; ++j) sum += ans[j] * a[i][j];
            if (abs(sum - a[i][m]) > EPS) return 0;
        }
        for (int i = 0; i < m; ++i)
            if (where[i] == -1) return INF;
        return 1;
    }
}

```

1.5.3 Matrix Exponentiation

```

const ll N = 2e5 + 1;
using Matrix = vector<vector<long long int>>;
Matrix multiply(const Matrix &A, const Matrix &B, long long MOD = -1) {
    // A = n x p
    // B = p x m
    int n = A.size();
    int m = B[0].size();
    int p = B.size();
    Matrix C(n, vector<long long>(m, 0));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            for (int k = 0; k < p; ++k) {
                C[i][j] += A[i][k] * B[k][j];
                if (MOD != -1) C[i][j] %= MOD;
            }
        }
    }
    return C;
}
// Identity matrix of size n

```



```
Matrix identity(ll n) {
    Matrix I(n, vector<long long>(n, 0));
    for (ll i = 0; i < n; i++) I[i][i] = 1;
    return I;
}
// Fast exponentiation of matrix A^power with optional modulo
Matrix matrix power(Matrix A, ll power, ll MOD=-1){
    ll n = A.Size();
    Matrix result = identity(n);
    while(power>0) {
        if(power&1){
            result = multiply(result, A, MOD);
        }
        A = multiply(A, A, MOD);
        power/=2;
    }
    return result;
}
```

1.5.4 Mobius Precomputation

```
const ll N=3e6+5;
// call SIEVE() and PRIMES() in int main()
vector<bool> sieve(N,true);
vector<ll> primes;
vector<ll> mu(N,1);
void Build_mu(){
    mu[1]=1;
    for(auto p:primes){
        ll sq=p*p;
        for(ll j=sq;j<N;j+=sq){
            mu[j]=0;
        }
        for(auto p:primes){
            for(ll j=p;j<N;j+=p)mu[j]*=-1;
        }
    }
}
```

1.5.5 Modular Inverse

```
ll inv(ll a, ll m){
    if(a==1) return a;
    return m - ((ll)(m/a)*inv(m%a,m)%m);
}
ll A_by_B mod(ll a, ll b){
    return (a*(binpow(b,M-2,M)))%M;
}
```

1.5.6 Nth Root Floor

```
ll Floor_nth_Root(ll x, ll n){
    if (x==0 || x==1) return x;
    auto power=[](ll a, ll exp){
        ll res=1;
        for(ll i=0;i<exp;i++)res*=a;
        return res;
    };
    ll v=power(2,63/n);
    ll lo=0,hi=min<ll>(v, ll(x+1));
    while (hi-lo>1){
        ll mid=(lo+hi)/2;
        ll count=n,val=1;
        while(count-->0)val*=mid;
        if(val<=x) lo=mid;
        else hi=mid;
    }return lo;
}
```

1.5.7 Sieve

```
// call SIEVE() and PRIMES() in the int main()
const ll N=1e6+5;
vector<bool> sieve(N,true);
vector<ll> primes;
void SIEVE(){
    sieve[1]=false;
    for(ll i=2;i<N;i++){
        if(!sieve[i]) continue;
        for(ll j=i*i;j<N;j+=i){
            sieve[j]=false;
        }
    }
}
void PRIMES(){
    loop(i,2,N){

```

```
if(sieve[i]){
    primes.pb(i);
}
```

1.5.8 Totient Values

```
vector<ll> phi(N+1,0ll);
//call phi 1 to n(N) in int main
void phi_1_to_n(int n) {
    for(int i=0;i<=n;i++) phi[i]=i;
    for(int i=2;i<=n;i++){
        if(phi[i]!=i) continue;
        for(int j=i;j<=n;j+=i)phi[j]-=phi[j]/i;
    }
}
```

1.6 Segment Tree

1.6.1 Persistent Segtree

```
//Your task is to maintain a list of arrays which initially
//has
//a single array. You have to process the following types of
//queries:
//1.Set the value a in array k to x.
//2.Calculate the sum of values in range [a,b] in array k.
//3.Create a copy of array k and add it to the end of the
//list.
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef vector<ll> vll;
const int MX = 1e6 + 9;
int root[MX];
struct Node{
    int left = 0;
    int right = 0;
    ll val = 0;
};
Node nodes[MX * 40];
struct PerSegTree{
    int id = 0;
    int build(vll &a, int lx, int rx){
        int cur = ++id;
        if (lx == rx){
            nodes[cur].val = a[lx];
            return cur;
        }
        int mid = (lx + rx) / 2;
        nodes[cur].left = build(a, lx, mid);
        nodes[cur].right = build(a, mid + 1, rx);
        nodes[cur].val = nodes[nodes[cur].left].val + nodes[nodes[cur].right].val;
        return cur;
    }
    int update(int prev, int i, ll v, int lx, int rx){
        int cur = ++id;
        nodes[cur] = nodes[prev];
        if (lx == rx){
            nodes[cur].val = v;
            return cur;
        }
        int mid = (lx + rx) / 2;
        if (i <= mid)
            nodes[cur].left = update(nodes[cur].left, i, v, lx, mid);
        else
            nodes[cur].right = update(nodes[cur].right, i, v, mid + 1, rx);
        nodes[cur].val = nodes[nodes[cur].left].val + nodes[nodes[cur].right].val;
        return cur;
    }
};
```

```
ll getSum(int cur, int l, int r, int lx, int rx){
    if (rx < l || r < lx){
        return 0;
    }
    if (lx >= l && rx <= r){
        return nodes[cur].val;
    }
    int mid = (lx + rx) / 2;
    return getSum(nodes[cur].left, l, r, lx, mid) + getSum(nodes[cur].right, l, r, mid + 1, rx);
}
```

```
ll kth(int u, int v, int k, int lx, int rx, vector<ll> &sortedA){
    //0 based
    if(lx == rx) return sortedA[lx];
    int mid = (lx + rx) / 2;
    int cnt = nodes[nodes[v].left].val - nodes[nodes[u].left].val;
    if(k <= cnt)
        return kth(nodes[u].left, nodes[v].left, k, lx, mid, sortedA);
    else
        return kth(nodes[u].right, nodes[v].right, k - cnt, mid + 1, rx, sortedA);
}
```

```
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vll a(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    PerSegTree tree;
    int version_count = 1;
    root[version_count] = tree.build(a, 1, n);
    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {
            int k, idx; ll x;
            cin >> k >> idx >> x;
            // Update array k directly
            root[k] = tree.update(root[k], idx, x, 1, n);
        }
        else if (type == 2) {
            int k, l, r;
            cin >> k >> l >> r;
            // Query array k
            cout << tree.getSum(root[k], l, r, 1, n) << "\n";
        }
        else if (type == 3) {
            int k;
            cin >> k;
            // Create a new copy referencing array k's current state
            version_count++;
            root[version_count] = root[k];
        }
    }
    return 0;
}
```

1.6.2 Segment Tree Merge Sort Tree

```
const ll MAXN=3e4+10;
vector<ll> tree[4*MAXN];
void build(ll a[], ll node, ll search_l, ll search_r){
    // root node = 1
    // search_l and search_r is 0-indexed
    if(search_l==search_r){
        tree[node]=vector<ll>(1,a[search_l]);
    }else{
        ll mid=(search_l+search_r)/2;
        build(a, node*2, search_l, mid);
        build(a, node*2+1, mid+1, search_r);
        merge(tree[node*2].begin(), tree[node*2].end(), tree[node*2+1].begin(), tree[node*2+1].end(), back_inserter(tree[node]));
    }
}
ll VectorsValue(vector<ll>&v, ll k){
    // Here goes the algorithm for the value you want to extract
    ll n=v.size();
    auto it=upper_bound(v.begin(), v.end(), k);
    if(it==v.end()){
        return 0;
    }
}
```

```

}else{
    ll x=it-v.begin();
    return n-x;
}
}
ll query(ll node, ll search_l, ll search_r, ll l, ll r, ll val) {
    // root node = 1
    // initial search_l = 0, search_r = 0, l = 0 indexed left pointer (inclusive)
    // l = 0 indexed left pointer (inclusive)
    // r = 0 indexed right pointer (inclusive)
    if (l>r) return 0;
    if (l==search_l && r==search_r) {
        return VectorsValue(tree[node],val);
    }
    ll mid=(search_l+search_r)/2;
    return query(node*2,search_l,mid,l,min(r,mid),val) +
           query(node*2+1,mid+1,search_r,max(l,mid+1),r,val);
}

```

1.6.3 Segment Tree with Lazy Propagation

```

struct ST {
    /* * PURPOSE: Segment Tree for Range Sum Query (RSQ) with Range Add Update.
    * * INDEXING:
    * - 1-based indexing is recommended for consistency (indices 1 to n).
    * - 0-based is possible if you call build(1, 0, n-1), but 'a' array must match.
    * * USAGE EXAMPLES:
    * 1. DEFINITION & BUILD:
    * int n = 5;
    * for(int i=1; i<=n; i++) a[i] = input[i]; // Fill global array 'a'
    * ST st(n); // Initialize vectors
    * st.build(1, 1, n); // Build tree from global array 'a'
    * * 2. UPDATE (Range Add):
    * // Add 10 to all elements in range [2, 4]
    * st.upd(1, 1, n, 2, 4, 10);
    * * 3. QUERY (Range Sum):
    * // Get sum of elements in range [2, 4]
    * ll result = st.query(1, 1, n, 2, 4);
    */
    #define lc (n << 1)
    #define rc ((n << 1) | 1)
    vector<ll> t, lazy;
    int N;
    ST(int n = 0) { init(n); }
    void init(int n) {
        N = n;
        t.assign(4 * (N + 5), 0);
        lazy.assign(4 * (N + 5), 0);
    }
    inline void push(int n, int b, int e) {
        if (lazy[n] == 0) return;
        // NOTE: Logic below is for SUM.
        // For Min/Max, remove *(e-b+1) and just add lazy[n]
        t[n] += lazy[n] * (e - b + 1);
        if (b != e) {
            lazy[lc] += lazy[n];
            lazy[rc] += lazy[n];
        }
        lazy[n] = 0;
    }
    inline void pull(int n) {
        t[n] = t[lc] + t[rc]; // NOTE: Change to max(t[lc], t[rc]) for RMQ
    }
    void build(int n, int b, int e) {
        lazy[n] = 0;
        if (b == e) { t[n] = a[b]; return; }
        int mid = (b + e) >> 1;
        build(lc, b, mid);
    }
}

```

```

build(rc, mid + 1, e);
pull(n);
}
void upd(int n, int b, int e, int i, int j, ll v) {
    push(n, b, e);
    if (j < b || e < i) return;
    if (i <= b && e <= j) {
        lazy[n] += v;
        push(n, b, e);
        return;
    }
    int mid = (b + e) >> 1;
    upd(lc, b, mid, i, j, v);
    upd(rc, mid + 1, e, i, j, v);
    pull(n);
}
ll query(int n, int b, int e, int i, int j) {
    push(n, b, e);
    if (j < b || e < i) return 0; // NOTE: Return infinity for Min, -infinity for Max
    if (i <= b && e <= j) return t[n];
    int mid = (b + e) >> 1;
    return query(lc, b, mid, i, j) + query(rc, mid + 1, e, i, j);
}
}

```

1.6.4 SegmentTree

```

ll segtree[N*4];
ll a[N];
ll func(ll a,ll b){
    return min(a,b);
}
void build segtree(ll node, ll lo, ll hi){
    if(lo==hi){
        segtree[node]=a[lo];
        return;
    }
    ll mid=(lo+hi)/2;
    build segtree(node*2,lo,mid);
    build segtree((node*2)+1,mid+1,hi);
    segtree[node]=func(segtree[node*2],segtree[(node*2)+1]);
}
ll query(ll node, ll lo, ll hi, ll l, ll r){
    if(r<lo || l>hi){
        return INF;
    }
    if(l<=lo && r>=hi){
        return segtree[node];
    }
    ll mid=(lo+hi)/2;
    ll left=query(node*2,lo,mid,l,r);
    ll right=query((node*2)+1,mid+1,hi,l,r);
    return func(left,right);
}
void update(ll node, ll lo, ll hi, ll pos, ll u){
    if(lo==hi){
        segtree[node]=u;
        return;
    }
    ll mid=(lo+hi)/2;
    if(pos<=mid){
        update(node*2,lo,mid,pos,u);
    }else{
        update(node*2+1,mid+1,hi,pos,u);
    }
    segtree[node]=func(segtree[node*2],segtree[node*2+1]);
}
int main(){
    ll n,q;
    cin>>n>>q;
    loop(i,1,n+1){
        cin>>a[i];
    }
    build segtree(1,1,n);
    loop(i,0,q){
        ll inp,l,r;
        cin>>inp>>l>>r;
        if(!(inp&1)){
            cout<<query(1,1,n,l,r)<<endl;
        }else{
            update(1,1,n,l,r);
        }
    }
}

```

```

}}}
1.7 String
1.7.1 Double String Hash
// string hash template
// call by
// prec(); in int main()
// Hashing h(s);
// pair<ll,ll> p=h.get_hash(l, r);(1 indexed)
const ll N = 1e6 + 9;
ll power(long long n, long long k, const ll mod) {
    ll ans = 1 % mod; n %= mod;
    if (n < 0) n += mod;
    while (k) {
        if (k & 1) ans = (long long) ans * n % mod;
        n = (long long) n * n % mod;
        k >>= 1; return ans;
    }
    const ll MOD1 = 127657753, MOD2 = 987654319;
    const ll p1 = 137, p2 = 277;
    ll ip1, ip2;
    pair<ll, ll> pw[N], ipw[N];
    void prec() {
        pw[0] = {1, 1};
        for (ll i = 1; i < N; i++) {
            pw[i].first = 1LL * pw[i - 1].first * p1 % MOD1;
            pw[i].second = 1LL * pw[i - 1].second * p2 % MOD2;
        }
        ip1 = power(p1, MOD1 - 2, MOD1);
        ip2 = power(p2, MOD2 - 2, MOD2);
        ipw[0] = {1, 1};
        for (ll i = 1; i < N; i++) {
            ipw[i].first = 1LL * ipw[i - 1].first * ip1 % MOD1;
            ipw[i].second = 1LL * ipw[i - 1].second * ip2 % MOD2;
        }
    }
    struct Hashing {
        ll n;
        string s; // 0 - indexed
        vector<pair<ll, ll>> hs; // 1 - indexed
        Hashing() {}
        Hashing(string s) {
            n = s.size();
            s = -s;
            hs.emplace back(0, 0);
            for (ll i = 0; i < n; i++) {
                pair<ll, ll> p;
                p.first = (hs[i].first + 1LL * pw[i].first * s[i] % MOD1) % MOD1;
                p.second = (hs[i].second + 1LL * pw[i].second * s[i] % MOD2) % MOD2;
                hs.push back(p);
            }
            pair<ll, ll> get_hash(ll l, ll r) { // 1 - indexed
                //assert(1ll <= l && l <= r && r <= n);
                pair<ll, ll> ans;
                ans.first = (hs[r].first - hs[l - 1].first + MOD1) * 1LL * ipw[l - 1].first % MOD1;
                ans.second = (hs[r].second - hs[l - 1].second + MOD2) * 1LL * ipw[l - 1].second % MOD2;
                return ans;
            }
            pair<ll, ll> get_hash() {
                return get_hash(1, n);
            }
        };
    }
    1.7.2 KMP and Z Function
vector<ll> KMP(string&s){
    ll n=(ll)s.size();vector<ll> pi(n);
    for (int i=1;i<n;i++) {
        int j=pi[i-1];
        while(j>0 && s[i]!=s[j]) j=pi[j-1];
        if(s[i]==s[j])j++;
        pi[i]=j;
    }return pi;
}
vector<ll> z function(string&s){
    ll n=s.size();
    vector<ll> z(n);
}

```

```

ll l=0,r=0;
for(ll i=1;i<n;i++){
    if(i<r) z[i]=min(r-i,z[i-l]);
    while(i+z[i]<n && s[z[i]]==s[i+z[i]]) z[i]++;
    if(i+z[i]>r){l=i;r=i+z[i];}
}return z;}

```

1.7.3 Suffix Array

```

// P[i][j] holds the ranking of the j-th suffix
// after comparing the first 2^i characters
// of that suffix
struct ranking{ll index;ll first;ll second;};
ll P[20][1000000];
bool comp(const ranking&a,const ranking&b) {
    if (a.first==b.first) return a.second<b.second;
    return a.first<b.first;}

vector<ll> build suffix array(const string s) {
    const ll n=s.length();
    const ll lgn=ceil(log2(n));
    vector<ll> sa(n);
    vector<ranking> ranks(n);
    ll i,j,step = 1;
    for(j=0;j<n;j++) P[0][j]=s[j]-'a';
    for(i=1;i<=lgn;i++,step++) {
        for(j=0;j<n;j++){
            ranks[j].index=j; ranks[j].first=P[i-1][j];
            ranks[j].second=(j+(1<=(i-1)<n)?P[i-1][j+(ll)
                ((1<=(i-1))):~1;
            }
            sort(ranks.begin(),ranks.end(),comp);
            for(j=0;j<n;j++){
                P[i][ranks[j].index]=(j>0&&ranks[j].first==ranks[
                    j-1].first
                    &&ranks[j].second==ranks[j-1].second)?P[i][
                        ranks[j-1].index]:j;
            }
            step*=2;
            for(i=0;i<n;i++) sa[P[step][i]]=i;
        }return sa;}

vector<ll> build lcp(const vector<ll>& sa,ll n) {
    vector<ll> lcp(n,0);
    ll step=ceil(log2(n));
    for(ll k=1;k<n;k++) {
        ll u=sa[k],v=sa[k-1];
        for(ll x=step;x>=0;x--) {
            if(u<n && v<n && P[x][u]==P[x][v]) {
                ll delta=(1ll<x);
                lcp[k]+=delta;
                u+=delta;v+=delta;}}return lcp;}

ll find_pattern_first_occurence(const string&text,const
    string&pattern,const vector<ll>&sa){
    ll lo=0,hi=(ll)sa.size()-1;
    ll res=(ll)sa.size();
    while(lo<=hi){
        ll mid=lo+(hi-lo)/2;
        if(text.compare(sa[mid],pattern.size(),pattern)>=0){
            res=mid;hi=mid-1;
        }else lo=mid+1;
    }return res;}

ll find_pattern_last_occurence(const string&text,const string
    &pattern,const vector<ll>&sa){
    ll lo=0,hi=(ll)sa.size()-1;
    ll res=-1;
    while(lo<=hi){
        ll mid=lo+(hi-lo)/2;
        if(text.compare(sa[mid],pattern.size(),pattern)<=0){
            res=mid;lo=mid+1;
        }else hi=mid-1;
    }return res;}

bool pattern_exists(const string&text,const string&pattern,
    const vector<ll>&sa,ll idx=-1){

```

```

if(idx==-1) idx=find_pattern_first_occurence(text,pattern
    ,sa);
if(idx==sa.size())return false;
if(text.compare(sa[idx],pattern.size(),pattern)==0)
    return true;
else return false;}

```

1.7.4 Trie

```

/**
 * TRIE (PREFIX TREE) TEMPLATE
 *
 * USE CASE:
 * - Fast string insertion, search, and prefix counting.
 * - O(L) complexity for all operations (L = string length).
 *
 * QUICK GUIDE:
 * - insert("apple"): Adds "apple". Increments .pass for 'a',
 *   'p', 'p', 'l', 'e' and .end for 'e'.
 * - find("apple"): Returns true ONLY if "apple" was
 *   explicitly inserted.
 * - prefixmatchcount("app"): Returns count of ALL strings
 *   starting with "app" (e.g., apple, apply).
 * - erase("apple"): Removes ONE instance. Cleanly deletes
 *   unused nodes.
 *
 * CAUTION:
 * - id(char c) assumes 'a'-'z'. Inputting 'A' or '1' will
 *   SEGFAULT.
 * - Memory: Each node uses 26 * 8 bytes (pointers). Space
 *   grows fast.
 */
struct Trie{
    static const ll SIG = 26;
    struct Node{
        Node *nxt[SIG];
        ll pass; // how many strings pass through this node
        ll end; // how many strings end at this node
        Node():pass(0),end(0){memset(nxt,0,sizeof(nxt));}
    };
    Node *root;
    Trie() {root = new Node();}
    ~Trie() {clear(root);}
    static inline ll id(char c) { return c - 'a'; } //make sure
    letter is lowercase or witness segfault
    void insert(const string &s){
        Node* cur=root;
        cur->pass++;
        for(char c:s){
            ll k=id(c);
            if(!cur->nxt[k]) cur->nxt[k] = new Node();
            cur=cur->nxt[k];
            cur->pass++;
        }
        cur->end++;
    }
    bool find(const string &s) const{
        const Node *cur = root;
        for(char c:s){
            ll k=id(c);
            if(!cur->nxt[k])return false;
            cur=cur->nxt[k];
        }
        return (cur->end)>0;
    }
    ll prefixmatchcount(const string &s) const{
        const Node *cur = root;
        for(char c:s){
            ll k=id(c);
            if(!cur->nxt[k]) return 0;
            cur=cur->nxt[k];
        }
        return cur->pass;
    }
    // erase ONE occurrence of s; returns true if something was
    // erased
    bool erase(const string &s){
        if (!find(s))return false;

```

```

Node* cur = root;
(cur->pass)--;
vector<pair<Node*, ll>> path; // (parent, edge_index)
path.reserve(s.size());
for(char c:s){
    ll k=id(c);
    path.push_back({cur, k});
    cur=cur->nxt[k];
    cur->pass--;
}
cur->end--;
// cleanup nodes from bottom if their pass becomes 0
for (ll i = (ll)path.size() - 1; i >= 0; i--){
    Node* parent = path[i].first;
    ll k = path[i].second;
    Node* child = parent->nxt[k];
    if (child->pass==0){
        clear(child);
        parent->nxt[k] = nullptr;
    }else{
        break; // higher nodes still needed
    }
    return true;}
private:
void clear(Node *cur){
    if (!cur)return;
    for (ll i = 0; i < SIG; i++)clear(cur->nxt[i]);
    delete cur;};

```

1.8 CUSTOM HASH

```

struct custom hash {
    static uint64 t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::
            steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};
// use : map<ll,ll,custom_hash>
// It also works for unordered_set!
// unordered_set<ll, custom_hash> s;

```

1.9 Coordinate Compression

```

// [1000, 5, 1000, 42] becomes [3, 1, 3, 2]
vector<ll> distinct colors;
for(ll i=0;i<n;i++){
    cin>>color[i];
    distinct_colors.push_back(color[i]);
}
sort(all(distinct_colors));
distinct_colors.erase(unique(all(distinct_colors)),
    distinct_colors.end());
for(ll i=0;i<n;i++){
    color[i]=lower_bound(all(distinct_colors),color[i])-
        distinct_colors.begin()+1;}

```

1.10 Custom_Comparator_for_pq

```

auto cmp = [](const pair<int, int>& a, const pair<int, int>&
    b) {
    return a.first > b.first; // Min-heap based on 'first'
};
priority_queue<pair<int, int>, vector<pair<int, int>>,
    decltpe(cmp)> pq(cmp);

```


1.11 RMQ

```
template <class T> struct RMQ{
    vector<vector<T>>> jmp;
    RMQ(const vector<T> &V):jmp(1,V){
        ll n=V.size();
        for(ll pw=1,k=1;pw*2<=n;pw*=2,++k) {
            jmp.emplace_back(n-pw*2+1);
            for(ll j=0;j<((ll)jmp[k].size());++j) {
                jmp[k][j]=min(jmp[k-1][j],jmp[k-1][j+pw]);
            }
        }
        // Queries the minimum in the inclusive range [a, b]
        T query(ll a, ll b){
            assert(a <= b);
            int dep=31-__builtin_clz(b-a+1);
            return min(jmp[dep][a],jmp[dep][b-(1<<dep)+1]);};
    }
```

1.12 Subtree queries - Number of nodes with color k in the subtree

```
const ll INF=1e16;
const double EPS = 1e-9;
const ll M=1e9+7;
const ll N=1e5+5;
set<ll> collec[N];
vector<ll> ans(N,-1);
vector<ll> graph[N];
vector<ll> euler(N,-1);
vector<ll> subtree_size(N,-1);
vector<ll> position_in_euler(N,-1);
vector<ll> heavy_child(N,-1);
ll idx=0;
vector<map<ll,ll>> answers(N);
vector<pll> qs;
vector<ll> color(N,-1);
vll cnt(N);
void dfs(ll node,ll par,bool keep){
    for(auto child:graph[node]){
        if(child!=par && child!=heavy_child[node]){
            dfs(child,node,false);
        }
    }
    if(heavy_child[node]!=-1) dfs(heavy_child[node],node,true);
    cnt[color[node]]++;
    for(auto child:graph[node]){
        if(child!=par && child!=heavy_child[node]){
            for(int i=position_in_euler[child];i<
                position_in_euler[child]+subtree_size[child];i++){
                ll vertex=euler[i];
                cnt[color[vertex]]++;};
    }
    for(auto &[a,b]:answers[node]){
        b=cnt[a];
    }
    if(!keep){
        for(int i=position_in_euler[node];i<position_in_euler
            [node]+subtree_size[node];i++){
            ll vertex=euler[i];
            cnt[color[vertex]]--;};
    }
}
void solve(){
    ll n,q;cin>>n>>q;
    vector<ll> distinct_colors;
    for(ll i=1;i<=n;i++){
        cin>>color[i];
        distinct_colors.push_back(color[i]);
    }
    sort(all(distinct_colors));
    distinct_colors.erase(unique(all(distinct_colors)),
        distinct_colors.end());
    for(ll i=1;i<=n;i++){
        color[i]=lower_bound(all(distinct_colors),color[i])-
            distinct_colors.begin()+1;
    }
    loop(i,0,n-1){
        ll a,b;
        cin>>a>>b;
        graph[a].push_back(b);
        graph[b].push_back(a);
    }
    while(q--){
```

```
ll a,b;
cin>>a>>b;
ll realb=lower_bound(all(distinct_colors),b)-
    distinct_colors.begin()+1;
    answers[a][realb]=-1;
    qs.push_back({a,realb});
    dfs_plus(1,-1);
    dfs(1,-1,false);
    for(auto [a,b]:qs){
        cout<<answers[a][b]<<endl;}}
    }
```

```
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    solve();}
```

1.13 bitset_operations

```
#include <iostream>
#include <bitset>
using namespace std;
int main() {
    bitset<8> b1(string("10101100")); // binary: 10101100
    bitset<8> b2(string("00111100")); // binary: 00111100
    cout << "b1 = " << b1 << endl; // Output: 10101100
    cout << "b2 = " << b2 << endl; // Output: 00111100
    cout << "\nb1 & b2 = " << (b1 & b2) << endl; // Output:
        00101100
    cout << "b1 | b2 = " << (b1 | b2) << endl; // Output:
        10111100
    cout << "b1 ^ b2 = " << (b1 ^ b2) << endl; // Output:
        10010000
    cout << "~b1 = " << (~b1) << endl; // Output:
        01010011
    bitset<8> b3 = b1;
    b3.set(); // Set all bits
    cout << "\nb3.set() = " << b3 << endl; // Output:
        11111111
    b3.reset(); // Reset all bits
    cout << "\nb3.reset() = " << b3 << endl; // Output:
        00000000
    b3 = b1;
    b3.flip(); // Flip all bits
    cout << "\nb3.flip() = " << b3 << endl; // Output:
        01010011
    b3.set(2); // Set a specific bit (index 2)
    cout << "\nb3.set(2) = " << b3 << endl; // Output:
        01010111
    b3.reset(2); // Reset a specific bit (index 2)
    cout << "\nb3.reset(2) = " << b3 << endl; // Output:
        01010011
    b3.flip(0); // Flip a specific bit (index 0)
    cout << "\nb3.flip(0) = " << b3 << endl; // Output:
        01010010
    // Count number of set bits
    cout << "\nb1.count() = " << b1.count() << endl; //
        Output: 4
    // Get value of a specific bit (index 3)
    cout << "\nb1[3] = " << b1[3] << endl; //
        Output: 1
    // Convert bitset to unsigned long
    cout << "b1.to_ulong() = " << b1.to_ulong() << endl; //
        Output: 172
    // Convert bitset to unsigned long long
    cout << "b1.to_ullong() = " << b1.to_ullong() << endl; //
        Output: 172
    // Convert bitset to string
    cout << "b1.to_string() = " << b1.to_string() << endl; //
        Output: 10101100
    return 0;
}
```

1.14 next permutation

```
int main() {
    vector<int> nums = {1, 2, 3};
```

```
// Sort first to ensure we get every single permutation
sort(nums.begin(), nums.end());
do{
    for(int n:nums) {
        cout << n << " ";
        cout << "\n";
    }
}while(next_permutation(nums.begin(),nums.end()));
return 0;}
```

2 Kabya

2.1 crt

```
#include <iostream>
#include <vector>
using namespace std;
// Chinese Remainder Theorem
int chineseRemainder(const vector<int>& num, const vector<int>
    & rem) {
    int prod = 1;
    for (int n : num) prod *= n;
    int result = 0;
    for (int i = 0; i < num.size(); i++) {
        int pp = prod / num[i];
        result += rem[i] * modInverse(pp, num[i]) * pp;
    }
    return result % prod;
}
int main() {
    vector<int> num = {3, 4, 5}; // moduli
    vector<int> rem = {2, 3, 1}; // remainders
    cout << "x = " << chineseRemainder(num, rem) << endl;
    return 0;}
```

2.2 extendedEuclidean

```
typedef pair<int, int> pii;
#define x first
#define y second
pii extendedEuclid(int a, int b) { // returns x, y | ax + by
    = gcd(a,b)
    if(b == 0) return pii(1, 0);
    else {
        pii d = extendedEuclid(b, a % b);
        return pii(d.y, d.x - d.y * (a / b));}
}
```

2.3 lcs

```
int lcs(string x, string y, int n, int m){
    int dp[n+1][m+1] = {};
    for (int i = 1; i <= n; i++){
        for (int j = 1; j <= m; j++){
            if (x[i-1] == y[j-1])
                dp[i][j] = 1 + dp[i-1][j-1];
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);}
    }
    return dp[n][m];}
```

2.4 millerRabin

```
typedef long long ll;
ll mulmod(ll a, ll b, ll c) {
    ll x = 0, y = a % c;
    while (b) {
        if (b & 1) x = (x + y) % c;
        y = (y << 1) % c;
        b >>= 1;
    }
    return x % c;
}
bool isPrime(ll n) {
    ll d = n - 1;
    int s = 0;
    while (d % 2 == 0) {
```

```
s++;d >= 1;}
// It's guranteed that these values will work for any
// number smaller than 3*10**18 (3 and 18 zeros)
int a[9] = { 2, 3, 5, 7, 11, 13, 17, 19, 23 };
for(int i = 0; i < 9; i++) {
    bool comp = binpow(a[i], d, n) != 1;
    if(comp) for(int j = 0; j < s; j++) {
        ll fp = binpow(a[i], (1LL << (ll)j)*d, n);
        if (fp == n - 1) {
            comp = false;
            break;}}
    if(comp) return false;
}
return true;}
```

2.5 zzsublime_build

```
{
"cmd" : ["g++ -std=c++14 $file_name -o $file_base name &&
        timeout 4s ./ $file_base_name<input.txt>output.txt"],
"selector" : "source.c",
"shell": true,
"working_dir" : "$file_path"
}
```

3 Sazid

3.1 Articulation_Bridge

```
// br = bridges, p = parent
vector<pair<int, int>> br;
int dfsBR(int u, int p) {
    low[u] = disc[u] = ++Time;
    for (int& v : adj[u]) {
        if (v == p) continue; // we don't want to go back through
        the same path.
        // if we go back is because we
        found another way back
        if (!disc[v]) { // if V has not been discovered before
            dfsBR(v, u); // recursive DFS call
            if (disc[u] < low[v]) // condition to find a bridge
                br.push_back({u, v});
            low[u] = min(low[u], low[v]); // low[v] might be an
            ancestor of u
        } else // if v was already discovered means that we found
        an ancestor
            low[u] = min(low[u], disc[v]); // finds the ancestor
            with the least discovery time
    }
}
void BR() {
    low = disc = vector<int>(adj.size());
    Time = 0;
    for (int u = 0; u < adj.size(); u++)
        if (!disc[u])
            dfsBR(u, u);
}
```

3.2 Articulation_Point

```
// adj[u] = adjacent nodes of u
// ap = AP = articulation points
// p = parent
// disc[u] = discovery time of u
// low[u] = 'low' node of u
int dfsAP(int u, int p) {
    int children = 0;
    low[u] = disc[u] = ++Time;
    for (int& v : adj[u]) {
        if (v == p) continue; // we don't want to go back through
        the same path.
        // if we go back is because we
        found another way back
```

```
if (!disc[v]) { // if V has not been discovered before
    children++;
    dfsAP(v, u); // recursive DFS call
    if (disc[u] <= low[v]) // condition #1
        ap[u] = 1;
    low[u] = min(low[u], low[v]); // low[v] might be an
    ancestor of u
} else // if v was already discovered means that we found
an ancestor
    low[u] = min(low[u], disc[v]); // finds the ancestor
    with the least discovery time
}
return children;
}
void AP() {
    ap = low = disc = vector<int>(adj.size());
    Time = 0;
    for (int u = 0; u < adj.size(); u++)
        if (!disc[u])
            ap[u] = dfsAP(u, u) > 1; // condition #2
}
```

4 Tamzid

4.1 bellmanford

```
struct Edge{
    ll from, to, cost;};
ll node, edges; // no. of node and edges in the graph
vector<Edge> vec(200005); // store the edges from,to and cost

vector<ll> dist(200005, INF);
vector<ll> par(200005, -1);
void Bellman_Ford(){
    dist[1] = 0;
    for (ll i = 1; i < node; i++){
        for (Edge it : vec){
            if (dist[it.from] < INF){
                if (dist[it.from] + it.cost < dist[it.to]){
                    dist[it.to] = max(dist[it.from] + it.cost, -INF);
                    par[it.to] = it.from;}}}}
}
void retrieve_shortest_path(vector<ll> &path){
    if (dist[node] == INF) return; // NO Path Exist
    for (ll cur = node; cur != -1; cur = par[cur]){
        path.push_back(cur);
    }
    reverse(path.begin(), path.end());
}
void find_negative_cycle(vector<ll> &path){
    // run this after bellman ford has been run
    ll start = -1;
    for (Edge it : vec){
        if (dist[it.from] < INF){
            if (dist[it.from] + it.cost < dist[it.to]){
                dist[it.to] = dist[it.from] + it.cost;
                start = it.to;
                par[it.to] = it.from;}}
    }
    if (start == -1) return; // no Negative cycle found so,
    path is empty
    for (ll times = 0; times < node; times++){
        start = par[start];
        path.push_back(start);
        for (ll cur = par[start]; cur = par[cur]){
            path.push_back(cur);
            if (cur == start) break;
        }
        reverse(path.begin(), path.end());
    }
}
```

4.2 moalgo

```
struct MO
{
    struct node
    {
        ll id, left, right;
    };
};
```

```
vector<node> queries;
vector<ll> val, cnt;
vector<ll> ans;

ll num, questions, block;

void build(vector<ll> &vec, vector<pair<ll, ll>> &q)
{
    num = vec.size();
    val.resize(num + 1);

    block = sqrt(num);
    // 1-based indexing
    for (ll i = 1; i <= num; i++)
        val[i] = vec[i - 1];

    questions = q.size();
    queries.resize(questions);
    ans.resize(questions);

    cnt = vec;
    sort(cnt.begin(), cnt.end());
    cnt.erase(unique(cnt.begin(), cnt.end()), cnt.end());
};

map<ll, ll> mp;
for (ll i = 0; i < cnt.size(); i++)
    mp[cnt[i]] = i;

for (ll i = 1; i <= num; i++)
    val[i] = mp[val[i]];

cnt.clear();
cnt.resize(mp.size());

for (ll i = 0; i < questions; i++)
{
    queries[i].id = i;
    queries[i].left = q[i].first;
    queries[i].right = q[i].second;
}

sort(queries.begin(), queries.end(), [&](node one,
node two)
{
    ll start1 = one.left / block, start2 = two.
    left / block;
    ll end1 = one.right / block, end2 = two.
    right / block;

    if (start1 == start2) return end1 < end2;
    return start1 < start2; });

ll times = 0;

void add(ll ind)
{
    if (!cnt[ind])
        times++;

    cnt[ind]++;
}

void del(ll ind)
{
    if (cnt[ind] == 1)
        times--;

    cnt[ind]--;
}

vector<ll> getResult()
{
    ll start = 1, end = 1;
    add(val[1]);

    for (ll i = 0; i < questions; i++)
    {
```

```

    ll left = queries[i].left, right = queries[i].right;

    while (end < right)
        add(val[++end]);
    while (start > left)
        add(val[--start]);

    while (end > right)
        del(val[end--]);
    while (start < left)
        del(val[start++]);

    ans[queries[i].id] = times;
}
return ans;
};

```

4.3 scckosaraju

```

ll num, edges;
vector<vector<ll>> adj, reverseadj;
vector<ll>visited, visitororder, ans;
vector<vector<ll>>allsc;

```

```

void dfs(ll node)
{
    visited[node]=1;
    for(auto it:adj[node])
    {
        if(!visited[it])dfs(it);
    }
    visitororder.push_back(node);
}

```

```

void dfs2(ll node,ll times)
{
    visited[node]=1;
    ans[node]=times;
    allsc[times-1].push_back(node);

    for(auto it:reverseadj[node])
    {
        if(!visited[it])
            dfs2(it,times);
    }
}

```

```

void kosaraju()
{
    for(ll i=0;i<num;i++)
    {
        if(!visited[i])dfs(i);
    }
    reverse(visitororder.begin(),visitororder.end());
    visited.assign(num,0);

    ll scc=0;
    for(auto i:visitororder)
    {
        if(!visited[i])
        {
            dfs2(i,++scc);
        }
    }

    cout<<scc<<endl;
    for(auto it:ans)cout<<it<<" ";

    for(ll i=0;i<scc;i++)
    {
        cout<<i<<": ";
        for(auto it:allsc[i])cout<<it<<" ";cout<<endl;
    }
}

```

4.4 sparsetable

```

struct sparsetable
{
    vector<vector<ll>> lookup;
    vector<ll> arr;
    ll times = 1, siz;

    void initialize(ll num)
    {
        siz = num;
        arr.resize(num);
        times = log2(num);
        lookup.resize(num, vector<ll>(times + 1));
    }

    ll merge(ll val1, ll val2)
    {
        return min(val1,val2);
    }

    void build(vector<ll> &vec)
    {
        for (ll i = 0; i < siz; i++)
            lookup[i][0] = vec[i];

        for (ll j = 1; j <= times; j++)
        {
            for (ll i = 0; i + (1LL <= j) <= siz; i++)
                lookup[i][j] = merge(lookup[i][j - 1], lookup[i + (1LL <= (j - 1))][j - 1]);
        }

        ll query(ll lo, ll hi)
        {
            ll j = log2(hi - lo + 1);

            return merge(lookup[lo][j], lookup[hi - (1LL <= j) + 1][j]);
        }
    }
};

```

4.5 treediameter

```

ll num,ans;
vector<vector<ll>>adj;
vector<bool>visited;
ll find_diameter(ll node){
    visited[node]=1;
    multiset<ll>ms;ms.insert(0);
    for(auto it:adj[node]){
        if(!visited[it]){
            ms.insert(find_diameter(it));
        }
    }
    auto it=ms.rbegin();
    ll maxi=*it;it--;
    ll second=*it;
    ans=max(ans,maxi+second);
    return maxi+1;
}

```

5 _D-One_

5.1 01_template

```

#include <bits/stdc++.h>
using namespace std;
#define int long long
using ll = long long;
using lld = long double;
const lld pi = acos(-1.0);
const ll INF = 4e18;
const ll mod = 1e9 + 7;
using pll = pair<int,int>;
using vll = vector<int>;
using vpll = vector<pair<int, int>>;

```

```

using vll = vector<vector<int>>;
using mll = map<int,int>;
using hashmap = unordered_map<int,int>;
#define ff first
#define ss second
#define pb push_back
#define lb lower_bound
#define ub upper_bound
#define all(v) (v).begin(), (v).end()
#define rall(v) (v).rbegin(), (v).rend()
#define print(v) for(auto &x:v) cout<<x<<" "; cout<<endl
#define sz(v) (int)(v).size()
#define endl '\n'
#define py cout<<"YES"<<endl
#define pn cout<<"NO"<<endl
#ifdef ONLINE_JUDGE
#define dbg(x) cerr << #x << " = " << x << '\n'
#else
#define dbg(x)
#endif

```

```

void D_One()
{
}

```

```

// Main
int32_t main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    cin >> t;
    while(t--) D_One();
    return 0;
}

```

5.2 02_lamda

```

function<return_type(types)> function_name = [&]( parameters ) -> return_type {
};

```

5.3 03_CheckFunctions

```

// vector<ll> primeFactors(int n){vll primes;while (n % 2 == 0){primes.pb(2);n = n / 2;}for (int i = 3; i <= sqrt(n); i = i + 2) {while (n % i == 0){primes.pb(i);n = n / i;}}if (n > 2)primes.pb(n);return primes;}
// bool isPowerOfTwo(int n){if(n==0)return false;return (ceil(log2(n)) == floor(log2(n)));}
// bool isPerfectSquare(ll x){if (x >= 0) {ll sr = sqrt(x); return (sr * sr == x);}return false;}
// string bin(int n){string s;while(n > 0){s.push_back('0' + (n & 1));n >>= 1;}reverse(s.begin(), s.end());return s;}
// ll dec(const string& s) {return stoll(s, nullptr, 2);}

```

5.4 04_import_Mod

```

int powerM(int a, int b, int m) {a %= m;int res = 1;while (b > 0) {if (b & 1)res = res * a % m;a = a * a % m;b >>= 1;}return res;}//log(n)

ll inv(ll a){return powerM(a, mod - 2, mod);}

int plusM(int a, int b){return ((a % mod) + (b % mod)) % mod;}

int intoM(int a, int b){return ((a % mod) * (b % mod)) % mod;}

int minusM(int a, int b){return ((a % mod) - (b % mod) + mod) % mod;}

int divideM(int a, int b){return ((a % mod) * inv(b) % mod) % mod;}

```

5.5 05_Comparator

```

//// je order e sort korte chai , oi order return kore dibo
// bool cmp(pll a, pll b){
//     if(a.ff != b.ff) return a.ff < b.ff; // first
//         increasing
//     return a.ss > b.ss; // second decresing
// }

```

5.6 06_Number_theory

```

// const int MAXN = 2e6 + 10;
// vector<bool> is_Prime(MAXN, true);
// vector<int> divs[MAXN];
// int spf[MAXN]; // Smallest Prime Factor
// void sieve()
// {
//     is_Prime[0] = is_Prime[1] = false;
//     for (int i = 2; i < MAXN; ++i)
//     {
//         if (is_Prime[i])
//         {
//             for (int j = i * i; j < MAXN; j += i)
//             {
//                 is_Prime[j] = false;
//                 if (spf[j] == 0)
//                     spf[j] = i;
//             }
//         }
//     }
// }
// void divisors()
// {
//     for (int i = 1; i < MAXN; ++i)
//         for (int j = i; j < MAXN; j += i)
//             divs[j].push_back(i);
// }
// vector<int> prime_factors(int n)
// {
//     vll ans;
//     while (n > 1)
//     {
//         int lp = spf[n];
//         if (is_Prime[n])
//         {
//             ans.pb(n);
//             break;
//         }
//         while (n % lp == 0)
//         {
//             n /= lp;
//             ans.pb(lp);
//         }
//     }
//     return ans;
// }

```

5.7 07_nCr

```

const int MAX = 2e5 + 10;
int fact[MAX], iFact[MAX];
void precompute() {
    fact[0] = 1;
    for (int i = 1; i < MAX; i++) fact[i] = (fact[i - 1] * i) % mod;
    iFact[MAX - 1] = powerM(fact[MAX - 1], mod - 2, mod);
    for (int i = MAX - 2; i >= 0; i--) iFact[i] = (iFact[i + 1] * (i + 1)) % mod;
}

```

```

//O(1)
int nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return (((fact[n] * iFact[r]) % mod) * iFact[n - r]) % mod;
}

```

5.8 08_Binary_Search

```

auto check = [&](int x) -> bool {
    return /* condition using mid */;
};
int low = 0, high = n - 1, ans = -1;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (check(mid)) {
        ans = mid;
        low = mid + 1;
    } else {
        high = mid - 1; // lower bound
    }
}

```

5.9 09_Kadane

```

int sum = 0, best = -1e18;
for(auto x : a){
    sum = max(x, sum + x);
    best = max(best, sum);
}
// best = max(best, 0LL); // allow empty subarray

```

5.10 10_2DPrefixSum

```

int n, m;
cin >> n >> m;
vector<vector<int>> a(n+1, vector<int>(m+1));
for(int i = 1; i <= n; i++)
    for(int j = 1; j <= m; j++)
        cin >> a[i][j];
vector<vector<int>> p(n+1, vector<int>(m+1, 0));
for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= m; j++) {
        p[i][j] = a[i][j]
            + p[i-1][j]
            + p[i][j-1]
            - p[i-1][j-1];
    }
}
auto query = [&](int x1, int y1, int x2, int y2) -> int {
    return p[x2][y2]
        - p[x1-1][y2]
        - p[x2][y1-1]
        + p[x1-1][y1-1];
};
int q;
cin >> q;
while(q--) {
    int x1, y1, x2, y2;
    cin >> x1 >> y1 >> x2 >> y2;
    cout << query(x1, y1, x2, y2) << "\n";
}

```

5.11 11_dfsTree

```

int n; cin >> n;
vector<vector<int>> adj(n + 10);
for (int i = 0; i < n - 1; ++i) {
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}
// vector<int> lvl(n + 10);
// vector<int> h(n + 10);
// vector<int> stree(n + 10, 1);
function<void(int, int)> dfs = [&](int u, int p) -> void {
    // lvl[u] = lvl[p] + 1; // If root is 0-indexed, set lvl[0] = -1
    for (int v : adj[u]) {
        if (v == p) continue;
        // lvl[v] = lvl[u] + 1;
        dfs(v, u);
        // h[u] = max(h[u], h[v] + 1);
        // stree[u] += stree[v];
    }
};
dfs(1, 0); // root = 1

```

5.12 12_dfsGraph

```

int n, m; cin >> n >> m;
vector<vector<int>> adj(n + 1);
vector<int> vis(n + 1, 0);
for (int i = 0; i < m; ++i) {
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}
function<void(int)> dfs = [&](int u) {
    vis[u] = 1;
    for (int v : adj[u]) {
        if (!vis[v])
            dfs(v);
    }
};
for (int i = 1; i <= n; ++i) {
    if (!vis[i]) {
        dfs(i);
    }
}

```

5.13 13_lca_blift

```

const int N = 200005;
const int LOG = 20;
int n;
cin >> n;
vector<vector<int>> adj(n + 1);
vector<vector<int>> up(n + 1, vector<int>(LOG));
vector<int> depth(n + 1, 0);
for (int i = 0; i < n - 1; i++) {
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}
// DFS to fill up[] and depth[]
function<void(int,int)> dfs = [&](int u, int p) {
    up[u][0] = p;
    for (int i = 1; i < LOG; i++) {
        up[u][i] = up[ up[u][i-1] ][i-1];
    }
    for (int v : adj[u]) {

```



```

    if (v == p) continue;
    depth[v] = depth[u] + 1;
    dfs(v, u);
}
};

int root = 1;
dfs(root, root);

// Lift node u by k steps
auto lift = [&](int u, int k) {
    for (int i = 0; i < LOG; i++) {
        if (k & (1 << i)) {
            u = up[u][i];
        }
    }
    return u;
};

// Find LCA of u and v
function<int(int,int)> lca = [&](int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    u = lift(u, depth[u] - depth[v]);
    if (u == v) return u;
    for (int i = LOG - 1; i >= 0; i--) {
        if (up[u][i] != up[v][i]) {
            u = up[u][i];
            v = up[v][i];
        }
    }
    return up[u][0];
};

// Distance between u and v
auto dist = [&](int u, int v) {
    int L = lca(u, v);
    return depth[u] + depth[v] - 2 * depth[L];
};

// Find k-th node on path from u to v (0-based, u is 0-th)
auto kth_node_on_path = [&](int u, int v, int k) {
    int L = lca(u, v);
    int du = depth[u] - depth[L];
    if (k <= du) {
        return lift(u, k);
    }
    else {
        int dv = depth[v] - depth[L];
        return lift(v, du + dv - k);
    }
};

// Usage examples:
// int x = lca(u, v);
// int d = dist(u, v);
// int kth_node = lift(u, k);
// int ans = kth_node_on_path(u, v, k);
// u is ancestor of v if lca(u, v) == u

```

5.14 14_bfs

```

vector<int> dist(n + 1, -1); // distance from source
int src = 1; // starting node
dist[src] = 0; // dist[i] gives distance from src to i

queue<int> q;
q.push(src);

while(!q.empty()) {
    int u = q.front(); q.pop();
    for(int v : adj[u]) {
        if(dist[v] == -1) { // not visited
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}

// bfs on Grid

```

```

// -----
int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0}; // 4 directions

vector<vector<int>> dist(n, vector<int>(m, -1));
queue<pair<int,int>> q;
q.push({0,0});
dist[0][0] = 0;

while(!q.empty()) {
    auto [x,y] = q.front(); q.pop();
    for(int i=0;i<4;i++){
        int nx = x+dx[i], ny=y+dy[i];
        if(nx>=0 && nx<n && ny>=0 && ny<m && dist[nx][ny]
        ]==-1){ // not visited : dist[nx][ny]==-1
            dist[nx][ny] = dist[x][y]+1;
            q.push({nx, ny});
        }
    }
}

```

5.15 15_dijkstra

```

const int N = 1e5 + 5;
const long long INF = 1e18;

vector<pair<int, int>> adj_list[N];
int visited[N], parent[N];
int nodes, edges;
long long d[N];
void dijkstra(int src)
{
    for (int i = 1; i <= nodes; i++) d[i] = INF;
    d[src] = 0;
    priority_queue<pair<long long, int>> pq;
    pq.push({0, src});

    while (!pq.empty()) {
        pair<long long, int> head = pq.top(); pq.pop();
        int selected_node = head.second;
        if (visited[selected_node]) continue;
        visited[selected_node] = 1;

        for (auto [adj_node, edge_cst] : adj_list[
            selected_node]) {
            if (d[selected_node] + edge_cst < d[adj_node]) {
                d[adj_node] = d[selected_node] + edge_cst;
                parent[adj_node] = selected_node;
                pq.push({-d[adj_node], adj_node});
            }
        }
    }
}

int main() {
    cin >> nodes >> edges;
    for (int i = 0; i < edges; i++) {
        int u, v, w; cin >> u >> v >> w;
        adj_list[u].push_back({v, w});
        adj_list[v].push_back({u, w});
    }
    int src = 1;
    dijkstra(src);
}

```

5.16 16_dsu

```

/*
DSU dsu(n); // for n elements (1-indexed)
int root = dsu.find(x); // representative (root) of x
Check if Two Elements Are in the Same Component
if (dsu.same(x, y)) { // x and y are connected}
dsu.merge(x, y); // merges components of x and y, if
different

```

```

int size = dsu.get_size(x); // returns size of component
containing x
dsu.count(); // number of disjoint sets/connectec components
*/
struct DSU
{
    vector<int> par, rnk, _sz; int c;
    DSU(int n) : par(n + 1), rnk(n + 1, 0), _sz(n + 1, 1), c(
        n) {
        for (int i = 1; i <= n; ++i) par[i] = i;
    }
    int find(int i) {return (par[i] == i ? i : (par[i] = find
        (par[i])));}
    bool same(int i, int j) { return find(i) == find(j);}
    int get_size(int i) { return _sz[find(i)];}
    int count() { return c;}
    int merge(int i, int j) {
        if ((i = find(i)) == (j = find(j))) return -1;
        else --C;
        if (rnk[i] > rnk[j]) swap(i, j);
        par[i] = j; _sz[j] += _sz[i];
        if (rnk[i] == rnk[j]) rnk[j]++;
        return j;
    }
};

```

5.17 17_ordered_set

```

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

#define int long long

/*=====
GENERIC ORDERED SET "T" can be:
- int
- pair<int,int>
===== */

template<typename T>
using ordered_set = tree< T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update >;

// count elements strictly smaller than x
// s.order_of_key(x)

// get k-th smallest element (0-indexed)
// *s.find_by_order(k);

int32_t main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    ordered_set<int> s;

    return 0;
}

```

5.18 18_segTree_node

```

struct node
{
    int mini, cnt;
    node(int m = INF, int c = 1) {
        mini = m, cnt = c;
    }
};

struct SegTree
{
    int n;
    vector<node> tree;
    void init(int sz) { n = sz; tree.assign(4 * n, node());}
}

```

```

node merge(node a, node b){
    if (a.mini < b.mini) return a;
    else if (b.mini < a.mini) return b;
    else {
        node tmp(a.mini, a.cnt + b.cnt);
        return tmp;
    }
}

void build(vector<int> &a, int v, int tl, int tr){
    if (tl == tr){tree[v] = node(a[tl], 1);}
    else{
        int tm = (tl + tr) / 2;
        build(a, v * 2, tl, tm);
        build(a, v * 2 + 1, tm + 1, tr);
        tree[v] = merge(tree[v * 2], tree[v * 2 + 1]);
    }
}

void build(vector<int> &a) {build(a, 1, 0, n - 1);}

node query(int v, int tl, int tr, int l, int r){
    if (l == tl && r == tr) return tree[v];
    int tm = (tl + tr) / 2;
    if (r <= tm) return query(v * 2, tl, tm, l, r);
    if (l > tm) return query(v * 2 + 1, tm + 1, tr, l, r);
    ;
    node left = query(v * 2, tl, tm, l, tm);
    node right = query(v * 2 + 1, tm + 1, tr, tm + 1, r);
    return merge(left, right);
}

node query(int l, int r) { return query(1, 0, n - 1, l, r); }

void update(int v, int tl, int tr, int pos, int new_val)
{
    if (tl == tr) {
        tree[v] = node(new_val, 1);
    } else {
        int tm = (tl + tr) / 2;
        if (pos <= tm) update(v * 2, tl, tm, pos, new_val);
        else update(v * 2 + 1, tm + 1, tr, pos, new_val);
        tree[v] = merge(tree[v * 2], tree[v * 2 + 1]);
    }
}

void update(int pos, int new_val){update(1, 0, n - 1, pos, new_val);}

// template 0-based
// a[0] to a[n-1] original array for build
// tree[1] whole segment [0..n-1]
// tree[v] covers segment [tl..tr]
// [l, r] = range you want to query
// [tl, tr] = segment of current node

// Usage:
// SegTree st;
// st.init(n);
// st.build(a);

```

5.19 19_fenwick

```

// A[] has to be 1-indexed
// create BIT of n elements : BIT<ll> bit(n);
// MODE A (point-update + prefix/range-sum):
// add v to A[idx]: bit.upd(idx, v);
// sum of A[1..idx]: ll sum1 = bit.query(idx);
// sum of A[l..r]: ll sumLR = bit.query(l, r);
// MODE B (range-update + point-query):
// add v to every A[i] for i in [l..r]: bit.upd(l, r, v);
// current A[idx]: ll Ai = bit.query(idx);
// Note: Do NOT mix MODE A and MODE B on the same instance

```

```

template <class T> struct BIT{
    int n; vector<T> t;
    BIT() : n(0) {}
    BIT(int _n) { n = _n; t.assign(n + 1, 0); }

```

```

static int lowbit(int x) { return x & -x; }
T query(int i) const {
    T ans = 0;
    for (; i > 0; i -= lowbit(i))
        ans += t[i];
    return ans;
}

void add(int i, T val) {
    if (i <= 0) return;
    for (; i <= n; i += lowbit(i))
        t[i] += val;
}

T query(int l, int r) const {
    if (l > r) return 0;
    return query(r) - query(l - 1);
}

void add(int l, int r, T val) {
    add(l, val); add(r + 1, -val);
}
};

```

5.20 20_rangefenwick

```

// 1-indexed array, like normal BIT.
// add(l, r, val) range update
// sum(l, r) range query
// BIT_RURQ<int> bit(n);
// bit.add(2, 5, 10);
// cout << bit.sum(1, 4);

template <class T>
struct BIT_RURQ {
    int n; vector<T> t1, t2;
    BIT_RURQ() : n(0) {}
    BIT_RURQ(int _n) : n(_n), t1(n + 1, 0), t2(n + 1, 0) {}
    static int lb(int x) { return x & -x; }
    void upd(vector<T> &t, int i, T val) {
        for (; i <= n; i += lb(i)) t[i] += val;
    }
    T query(const vector<T> &t, int i) const {
        T res = 0;
        for (; i > 0; i -= lb(i)) res += t[i];
        return res;
    }
    void add(int l, int r, T val) {
        upd(t1, l, val); upd(t1, r + 1, -val);
        upd(t2, l, val * (l - 1)); upd(t2, r + 1, -val * r);
    }
    T prefix(int i) const {
        return i * query(t1, i) - query(t2, i);
    }
    T sum(int l, int r) const {
        return prefix(r) - prefix(l - 1);
    }
};

```

6 zFrom Sifat Bhai

6.1 Data Structure

6.1.1 persistant_segtree

```

/** Persistent Segment Tree using static Array
    Point Update, Range Sum
    Initialize ncnt to 0 in every test case */

```

```
const int MAX = 100010;
```

```
int ncnt = 0;
```

```
struct node {
    int sum;
    int left, right;
    node() {}
    node(int val) {
        sum = val;

```

```

        left = right = -1;
    }
} tree[ ? ];

// input array
int ara[MAX];
// root nodes for all versions
int version[MAX];

void build(int n, int st, int ed) {
    if (st == ed) {
        tree[n] = node(ara[st]);
        return;
    }
    int mid = (st + ed) / 2;
    tree[n].left = ++ncnt;
    tree[n].right = ++ncnt;
    build(tree[n].left, st, mid);
    build(tree[n].right, mid + 1, ed);
    tree[n].sum = tree[tree[n].left].sum + tree[tree[n].right].sum;
}

```

```

void update(int prev, int cur, int st, int ed, int id, int val)
{
    if (id > ed or id < st) return;
    if (st == ed) {
        tree[cur] = node(val);
        return;
    }
    int mid = (st + ed) / 2;
    if (id <= mid) {
        tree[cur].right = tree[prev].right;
        tree[cur].left = ++ncnt;
        update(tree[prev].left, tree[cur].left, st, mid, id, val);
    } else {
        tree[cur].left = tree[prev].left;
        tree[cur].right = ++ncnt;
        update(tree[prev].right, tree[cur].right, mid + 1, ed, id, val);
    }
    tree[cur].sum = tree[tree[cur].left].sum + tree[tree[cur].right].sum;
}

int query(int n, int st, int ed, int i, int j) {
    if (st >= i && ed <= j) return tree[n].sum;
    int mid = (st + ed) / 2;
    if (mid < i) return query(tree[n].right, mid + 1, ed, i, j);
    else if (mid >= j) return query(tree[n].left, st, mid, i, j);
    else return query(tree[n].left, st, mid, i, j) + query(tree[n].right, mid + 1, ed, i, j);
}

```

```

int main() {
    int n, q, l, r, k;
    sii(n, q);
    version[0] = ++ncnt;
    build(version[0], 1, n);
    version[1] = ++ncnt;
    update(version[0], version[1], 1, n, id, val);
    query(version[0], 1, n, id, id);
    query(version[1], 1, n, id, id);
    return 0;
}

```

6.2 Geometry

6.2.1 3D

6.2.1.1 3D Convex Hull

```
#include <bits/stdc++.h>

#define ll long long
#define sz(x) ((int) (x).size())
#define all(x) (x).begin(), (x).end()
#define vi vector<int>
#define pii pair<int, int>
#define rep(i, a, b) for(int i = (a); i < (b); i++)
using namespace std;
template<typename T>
using minpq = priority_queue<T, vector<T>, greater<T>>;

typedef long double ftype;

struct pt3 {
    ftype x, y, z;
    pt3(ftype x = 0, ftype y = 0, ftype z = 0) : x(x), y(y), z(z) {}
    pt3 operator-(const pt3 &o) const {
        return pt3(x - o.x, y - o.y, z - o.z);
    }
    pt3 cross(const pt3 &o) const {
        return pt3(y * o.z - z * o.y, z * o.x - x * o.z, x * o.y - y * o.x);
    }
    ftype dot(const pt3 &o) const {
        return x * o.x + y * o.y + z * o.z;
    }
};

struct face {
    int a, b, c;
    pt3 q;
};

const ftype EPS = 1e-9;

vector<face> hull3(const vector<pt3> &p) {
    int n = sz(p);
    assert(n >= 3);
    vector<face> f;
    vector<vector<bool>> dead(n, vector<bool>(n, true));
    auto add_face = [&](int a, int b, int c) {
        f.push_back({a, b, c, (p[b] - p[a]).cross(p[c] - p[a])});
        dead[a][b] = dead[b][c] = dead[c][a] = false;
    };
    add_face(0, 1, 2);
    add_face(0, 2, 1);
    rep(i, 3, n) {
        vector<face> f2;
        for(face &F : f) {
            if((p[i] - p[F.a]).dot(F.q) > EPS) {
                dead[F.a][F.b] = dead[F.b][F.c] = dead[F.c][F.a] = true;
            } else {
                f2.push_back(F);
            }
        }
        f.clear();
        for(face &F : f2) {
            int arr[3] = {F.a, F.b, F.c};
            rep(j, 0, 3) {
                int a = arr[j], b = arr[(j + 1) % 3];
                if(dead[b][a]) {
                    add_face(b, a, i);
                }
            }
            f.insert(f.end(), all(f2));
        }
    }
    return f;
}
```

6.2.2 Circle

6.2.2.1 Circle Intersection Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

```
#include "Point.h"
```

```
typedef Point<double> P;
bool circleInter(P a, P b, double r1, double r2, pair<P, P>* out) {
    if
```

```
if (a == b) { assert(r1 != r2); return false; }
P vec = b - a;
double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
       p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2
;
if (sum*sum < d2 || dif*dif > d2) return false;
P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2
);
*out = {mid + per, mid - per};
return true;
}
```

6.2.2.2 Circle Polygon Intersection Returns the area of the intersection of a circle with a ccw polygon. Time: $O(n)$

```
#include "Point.h"

typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
    auto tri = [&](P p, P q) {
        auto r2 = r * r / 2;
        P d = q - p;
        auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2
        ();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det))
        ;
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
        P u = p + d * s, v = p + d * t;
        return arg(p, u) * r2 + u.cross(v)/2 + arg(v, q) * r2;
    };
    auto sum = 0.0;
    rep(i, 0, sz(ps))
        sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
    return sum;
}
```

6.2.2.3 Circle Tangents Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

```
#include "Point.h"

template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
    {
        P d = c2 - c1;
        double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
        if (d2 == 0 || h2 < 0) return {};
        vector<pair<P, P>> out;
        for (double sign : {-1, 1}) {
            P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
            out.push_back({c1 + v * r1, c2 + v * r2});
        }
        if (h2 == 0) out.pop_back();
        return out;
    }
}
```

6.2.3 Polygon

6.2.3.1 Hull Diameter Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

```
#include "Point.h"
```

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
    rep(i, 0, j)
        for (; j = (j + 1) % n) {
            res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
            if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >=
                0)
                break;
        }
    return res.second;
}
```

6.2.3.2 Line Hull Intersection Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon:

- $(-1, -1)$ if no collision,
- $(i, -1)$ if touching the corner i ,
- (i, i) if along side $(i, i + 1)$,
- (i, j) if crossing sides $(i, i + 1)$ and $(j, j + 1)$.

In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.

Time: $O(\log n)$

```
#include "Point.h"
```

```
#define cmp(i, j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n
]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P> &poly, P dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m
        ;
    }
    return lo;
}
```

```
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P> &poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    rep(i, 0, 2) {
        int lo = endB, hi = endA, n = sz(poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;

```

```

    (cml(m) == cml(endB) ? lo : hi) = m;
}
res[i] = (lo + !cml(hi)) % n;
swap(endA, endB);
}
if (res[0] == res[1]) return {res[0], -1};
if (!cml(res[0]) && !cml(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
        case 0: return {res[0], res[0]};
        case 2: return {res[1], res[1]};
    }
return res;
}

```

6.2.3.3 Polygon Center Returns the center of mass for a polygon.
Time: $O(n)$

```
#include "Point.h"
```

```

typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
    P res(0, 0); double A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}

```

6.3 Graph

6.3.1 2Sat

```

/**
 * 1 based index for variables
 * F = (a op b) and (c op d) and ..... (y op z)
 * a, b, c ... are the variables
 * sat::satisfy() returns true if there is some assignment(True
 * /False)
 * for all the variables that make F = True
 * * init() at the start of every case
 */
namespace sat{
    const int MAX = 200010; // number of variables * 2
    bool vis[MAX];
    vector<int> ed[MAX], rev[MAX];
    int n, m, ptr, dfs_t[MAX], ord[MAX], par[MAX];
    inline int inv(int x){
        return ((x) <= n ? (x + n) : (x - n));
    }
    // Call init once
    void init(int vars){
        n = vars, m = vars << 1;
        for (int i = 1; i <= m; i++){
            ed[i].clear();
            rev[i].clear();
        }
        // Adding implication, if a then b ( a --> b )
        inline void add(int a, int b){
            ed[a].push_back(b);
            rev[b].push_back(a);
        }
        // (a or b) is true --> OR(a,b)
        // (¬a or b) is true --> OR(inv(a),b)
        // (a or ¬b) is true --> OR(a,inv(b))
        // (¬a or ¬b) is true --> OR(inv(a),inv(b))
        inline void OR(int a, int b){
            add(inv(a), b);
            add(inv(b), a);
        }
        // same rule as or
        inline void AND(int a, int b){
            add(a, b);
            add(b, a);
        }
    }
    // same rule as or

```

```

void XOR(int a,int b){
    add(inv(b), a);
    add(a, inv(b));
    add(inv(a), b);
    add(b, inv(a));
}
// same rule as or
inline void XNOR(int a, int b){
    add(a,b);
    add(b,a);
    add(inv(a), inv(b));
    add(inv(b), inv(a));
}
// (x <= n) means forcing variable x to be true
// (x = n + y) means forcing variable y to be false
inline void force true(int x){
    add(inv(x), x);
}
inline void topsort(int s){
    vis[s] = true;
    for(int x : rev[s]) if(!vis[x]) topsort(x);
    dfs_t[s] = ++ptr;
}
inline void dfs(int s, int p){
    par[s] = p;
    vis[s] = true;
    for(int x : ed[s]) if (!vis[x]) dfs(x, p);
}

void build(){
    CLR(vis);
    ptr = 0;
    for(int i=m;i>=1;i--) {
        if (!vis[i]) topsort(i);
        ord[dfs_t[i]] = i;
    }
    CLR(vis);
    for (int i = m; i >= 1; i--){
        int x = ord[i];
        if (!vis[x]) dfs(x, x);
    }
    // Returns true if the system is 2-satisfiable and returns
    // the solution (vars set to true) in vector res
    bool satisfy(vector<int>& res){
        build();
        CLR(vis);
        for (int i = 1; i <= m; i++){
            int x = ord[i];
            if (par[x] == par[inv(x)]) return false;
            if (!vis[par[x]]){
                vis[par[x]] = true;
                vis[par[inv(x)]] = false;
            }
            res.clear();
            for (int i = 1; i <= n; i++){
                if (vis[par[i]]) res.push_back(i);
            }
            return true;
        }
    }
}

```

6.4 Math

6.4.1 FloorSum

```

// 0(log a) sum^n floor(ax + b / c) = f
long long FloorSumAP(long long a, long long b, long long c,
    long long n){
    if(!a) return (b / c) * (n + 1);
    if(a >= c or b >= c) return ((n * (n + 1) ) / 2) * (a / c
        ) + (n + 1) * (b / c) + FloorSumAP(a % c, b % c, c, n);
    long long m = (a * n + b) / c;
    return m * n - FloorSumAP(c, c - b - 1, a, m - 1);
}
// 0(log a) sum^n x * floor(ax + b / c) = g, sum^n floor(ax +
    b / c)^2 = h
struct dat {
    long long f, g, h;
    dat(long long f = 0, long long g = 0, long long h = 0) : f(f), g(g), h(h) {}
};
long long mul(long long a, long long b){
    return (a * b) % MOD;
}
dat query(long long a, long long b, long long c, long long n)
{
    if(!a) return {mul(n + 1, b / c), mul(mul(mul(b / c, n), n
        + 1), inv2), mul(mul(n + 1, b / c), b / c)};
    long long f, g, h; dat nxt;
    if(a >= c or b >= c){

```

```

    nxt = query(a % c, b % c, c, n);
    f = (nxt.f + mul(mul(mul(n, n + 1), inv2), a / c) + mul(n
        + 1, b / c)) % MOD;
    g = (nxt.g + mul(a / c, mul(mul(n, n + 1), mul(2 * n + 1,
        inv6))) + mul(mul(b / c, mul(n, n + 1)), inv2)) % MOD;
    h = (nxt.h + 2 * mul(b / c, nxt.f) + 2 * mul(a / c, nxt.g
        ) + mul(mul(a / c, a / c), mul(mul(n, n + 1), mul(2 * n
        + 1, inv6))) + mul(mul(b / c, b / c), n + 1) + mul(mul(a
        / c, b / c), mul(n, n + 1))) % MOD;
    return {f, g, h};
}
long long m = (a * n + b) / c;
nxt = query(c, c - b - 1, a, m - 1);
f = (mul(m, n) - nxt.f) % MOD;
g = mul(mul(m, mul(n, n + 1)) - nxt.h - nxt.f, inv2);
h = (mul(n, mul(m, m + 1)) - 2 * nxt.g - 2 * nxt.f - f) %
    MOD;
return {f, g, h};
}

```

6.4.2 catalan

```

void init() {
    catalan[0] = catalan[1] = 1;
    for (int i=2; i<=n; i++) {
        catalan[i] = 0;
        for (int j=0; j < i; j++) {
            catalan[i] += (catalan[j] * catalan[i-j-1]) % MOD;
        }
        if (catalan[i] >= MOD) {
            catalan[i] -= MOD;
        }
    }
}

```

6.4.3 gen_all_k_combs

```

vector<int> ans;
void gen(int n, int k, int idx, bool rev) {
    if (k > n || k < 0)
        return;
    if (!n){
        for(int i = 0; i < idx; ++i) {
            if (ans[i])
                cout << i + 1;
            cout << "\n";
            return;
        }
        ans[idx]=rev;gen(n-1,k-rev,idx+1,false);
        ans[idx] = !rev;gen(n-1,k-!rev,idx+1,true);
    }
    void all_combinations(int n, int k) {
        ans.resize(n);
        gen(n, k, 0, false);
    }
}

```

6.4.4 next_lexicographical_k_comb

```

bool next_combination(vector<int>& a, int n) {
    int k = (int)a.size();
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - k + i + 1) {
            a[i]++;
            for (int j = i + 1; j < k; j++)
                a[j] = a[j - 1] + 1;
            return true;
        }
    }
    return false;
}

```

6.4.5 ntt

```

const int mod = 998244353;
const int root = 15311432;
const int k = 1 << 23;
int root_1;
vector<int> rev;
void pre(int sz){
    root_1 = bigmod(root, mod - 2, mod);
    if (rev.size() == sz) return;
    rev.resize(sz);
    rev[0] = 0;
    int lg_n = __builtin_ctz(sz);

```



```

    for (int i = 1; i < sz; ++i) rev[i]=(rev[i>>1]>>1)|((i&1)
    <<(lg_n-1));
}

void fft(vector<int> &a, bool inv){
    int n = a.size();
    for (int i = 1; i < n - 1; ++i) if (i < rev[i]) swap(a[i]
    , a[rev[i]]);
    for (int len = 2; len <= n; len <= 1) {
        int wlen = inv ? root_1 : root;
        for (int i = len; i < n; i <= 1) wlen = 1ll * wlen *
        wlen % mod;
        for (int st = 0; st < n; st += len){
            int w = 1;
            for (int j = 0; j < len / 2; j++){
                int ev = a[st + j];
                int od = 1ll * a[st + j + len / 2] * w % mod;
                a[st + j] = ev + od < mod ? ev + od : ev + od
                - mod;
            }
        }
    }
}

```

```

        a[st + j + len / 2] = ev - od >= 0 ? ev - od
        : ev - od + mod;
        w = 1ll * w * wlen % mod;
    }
}

if (inv){
    int n_1 = bigmod(n, mod - 2, mod);
    for (int &x : a) x = 1ll * x * n_1 % mod;
}

vector<int> mul(vector<int> &a, vector<int> &b){
    int n = a.size(), m = b.size(), sz = 1;
    while (sz < n + m - 1) sz <= 1;
    vector<int> x(sz), y(sz), z(sz);
    for (int i = 0; i < sz; ++i){
        x[i] = i < n ? a[i] : 0;
        y[i] = i < m ? b[i] : 0;
    }
    pre(sz);fft(x, 0);fft(y, 0);
    for (int i = 0; i < sz; ++i) z[i] = 1ll * x[i] * y[i] %
    mod;
    fft(z, 1);z.resize(n + m - 1);
}

```

```

    return z;
}

```

6.4.6 xor_basis

```

int basis[d]; // basis[i] keeps the mask of the vector whose
// value is i
int sz;
void insertVector(int mask) {
    for (int i = 0; i < d; i++) {
        if ((mask & 1 <= i) == 0) continue;
        if (!basis[i]) { // If there is no basis vector with the i'
            th bit set, then insert this vector into the basis
            basis[i] = mask;
            ++sz;return;
        }
        mask ^= basis[i]; // Otherwise subtract the basis vector
        from this vector
    }
}

```